Lecture # 1

Introduction/Overview of Course/Brief revision of Computer History

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

بسم الله الرحمن الرحيم

Parallel and Distributed Computing

Muhammad Zain Uddin
Lecturer,
IBA

Parallel and Distributed Computing

?

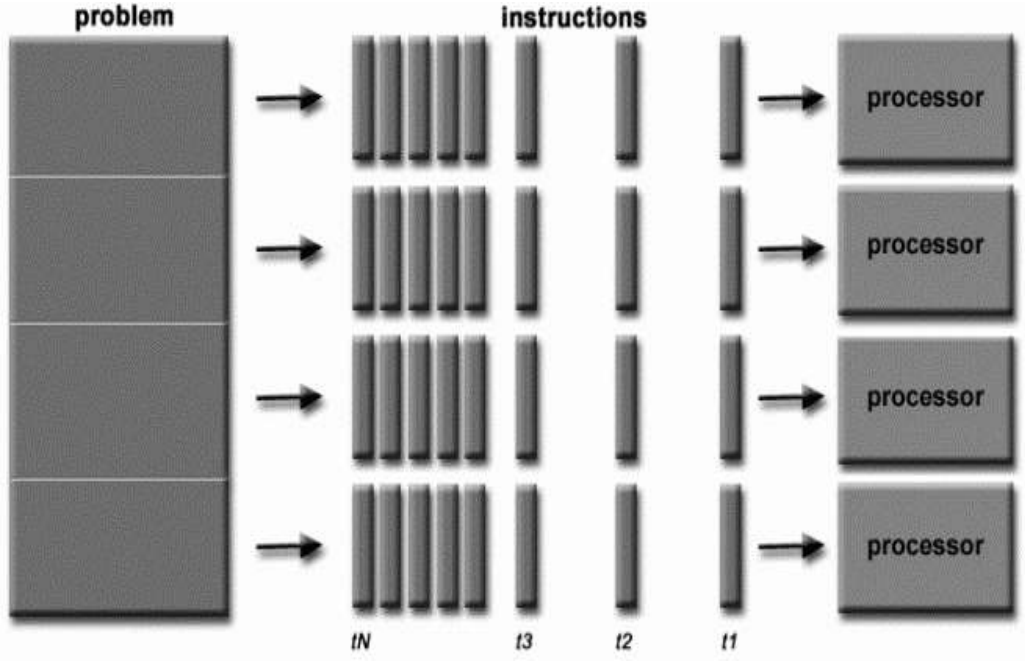
?

What is Parallel Computing

A collection of processing elements that communicate and cooperate to solve problems

But can a single processor perform parallel computing???.

YES

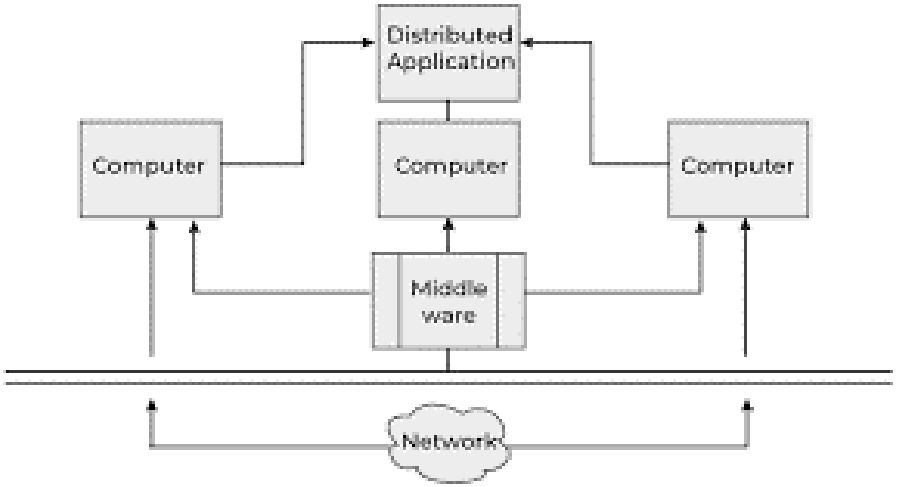


Ways of Parallel computing on single processor

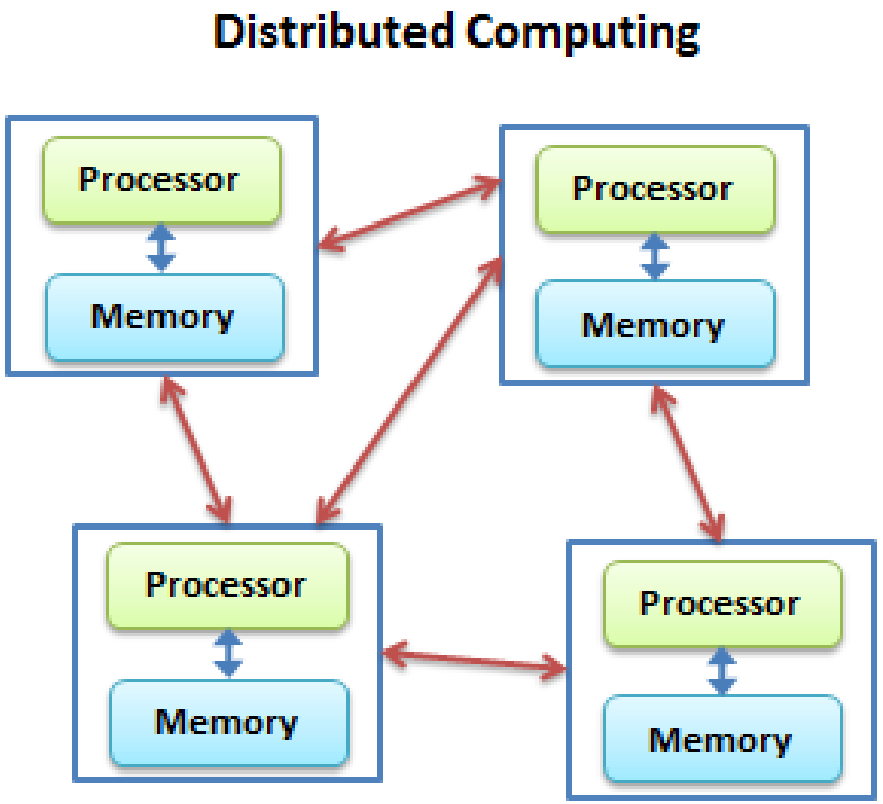
- Pipeline
- Multi threading
- Vector processing

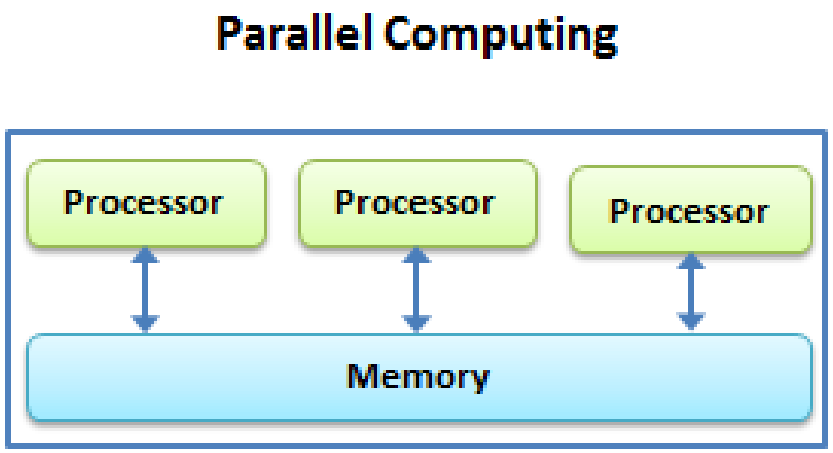
What is Distributed computing

A collection of independent computers that appear to its users as a single coherent system.



Parallel vs Distributed





Why go Parallel or Distributed?

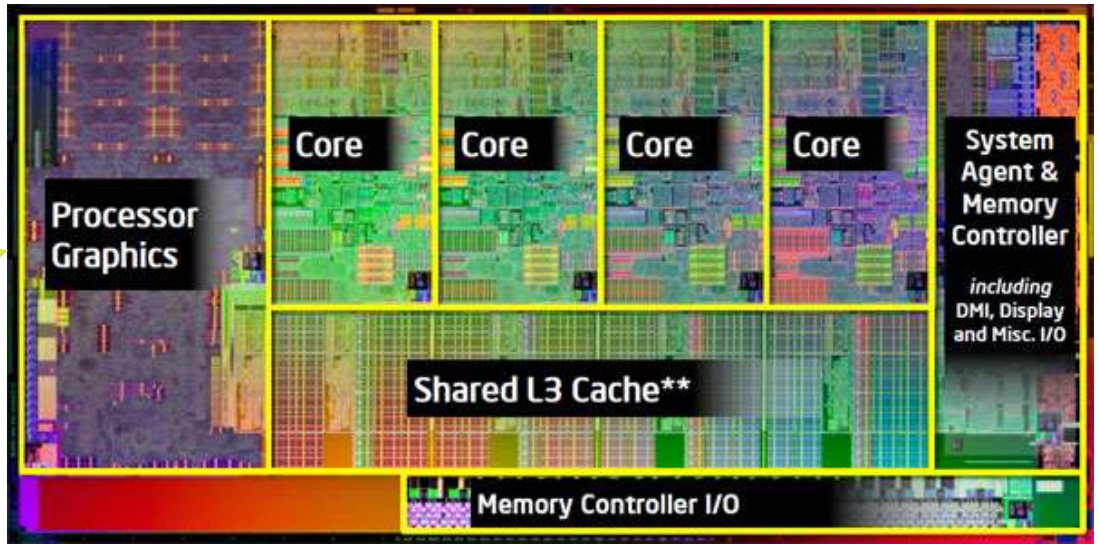
- Increased processing power:
  - "Parallel computing can achieve speedups of up to 1000 times for some applications." (Patterson & Hennessy, Computer Architecture, 5th Edition, p. 545)
- Improved efficiency:
  - "Parallel algorithms can sort a billion numbers in less than 10 seconds." (Quinn, Parallel Programming in C with MPI and OpenMP, p. 3)
- Ability to tackle big data:
  - "Parallel computing is essential for handling the massive datasets generated in fields like genomics and astronomy." (Tanenbaum & Steen, Distributed Systems, 3rd Edition, p. 851)
- Solving complex problems:
  - "Parallel computing has enabled breakthroughs in weather forecasting, drug discovery, and financial modeling." (Dongarra, et al., The International Exascale Software Project Roadmap, 2018, p. 5)

Parallel Computer Architectures

Several separate computers,
some means for communication
(e.g., Ethernet)

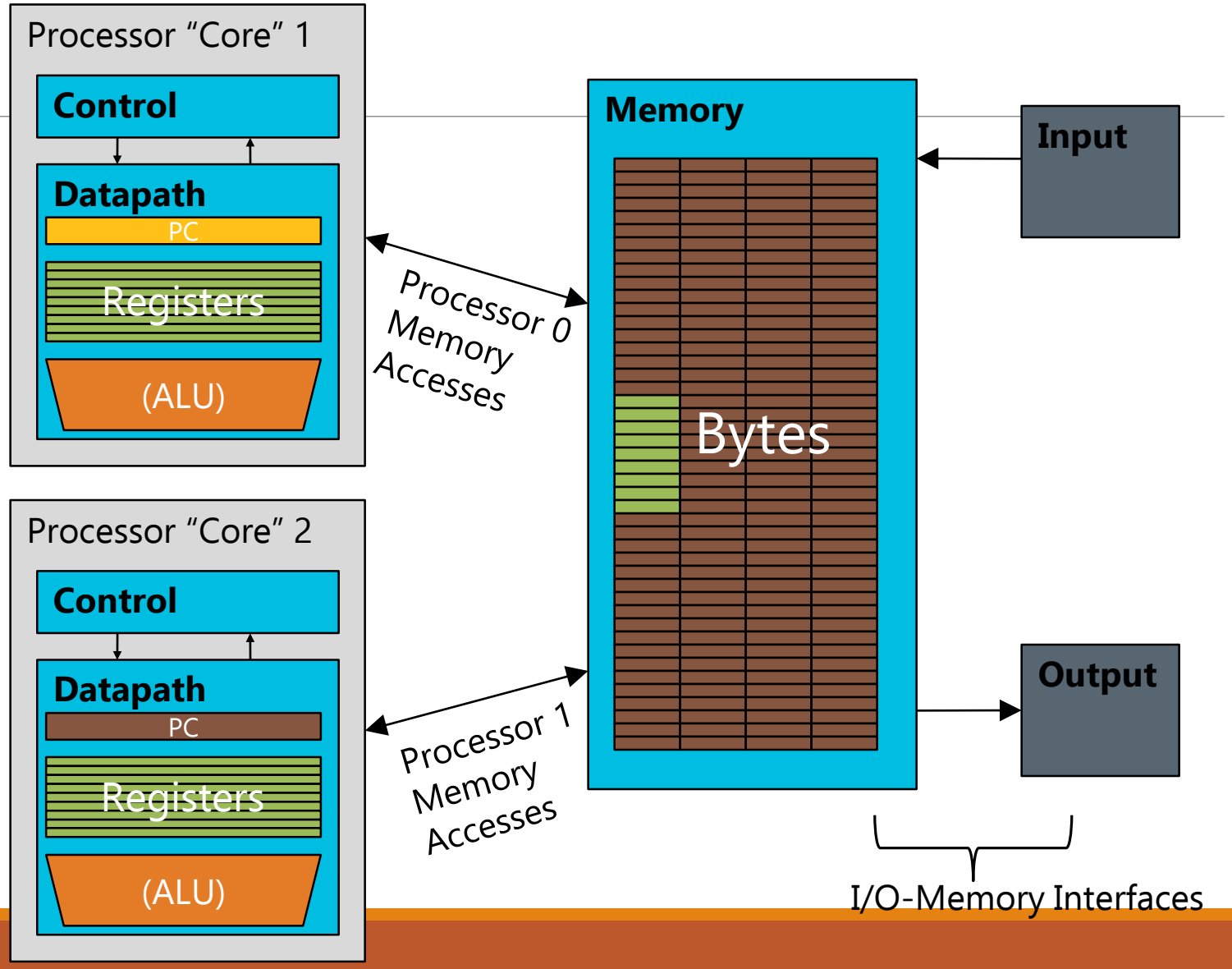
Massive array of computers,
fast communication between processors

GPU "graphics processing unit"

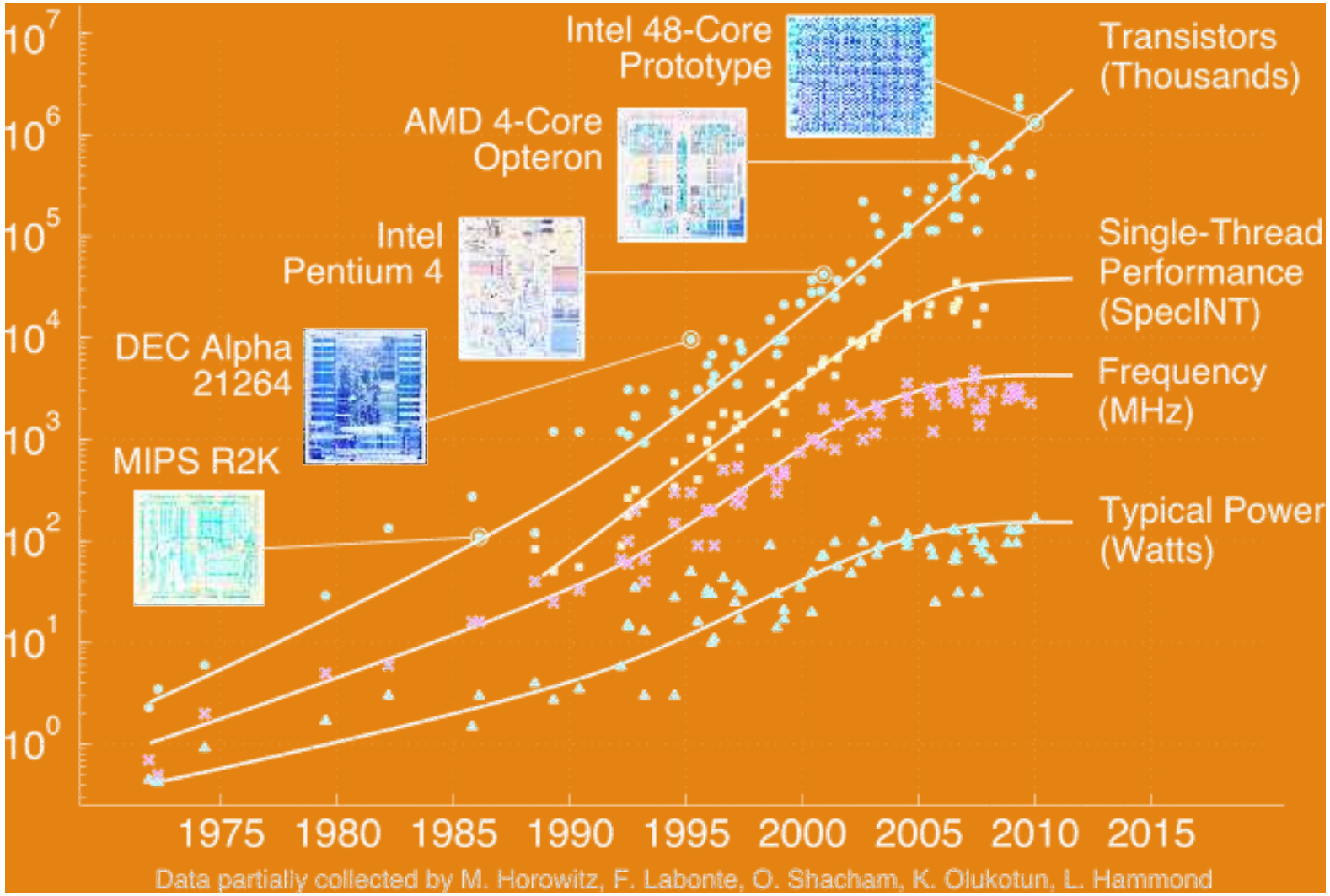


Multi-core CPU:
1 datapath in single chip
share L3 cache, memory, peripherals
Example: Hive machines

Example: CPU with Two Cores

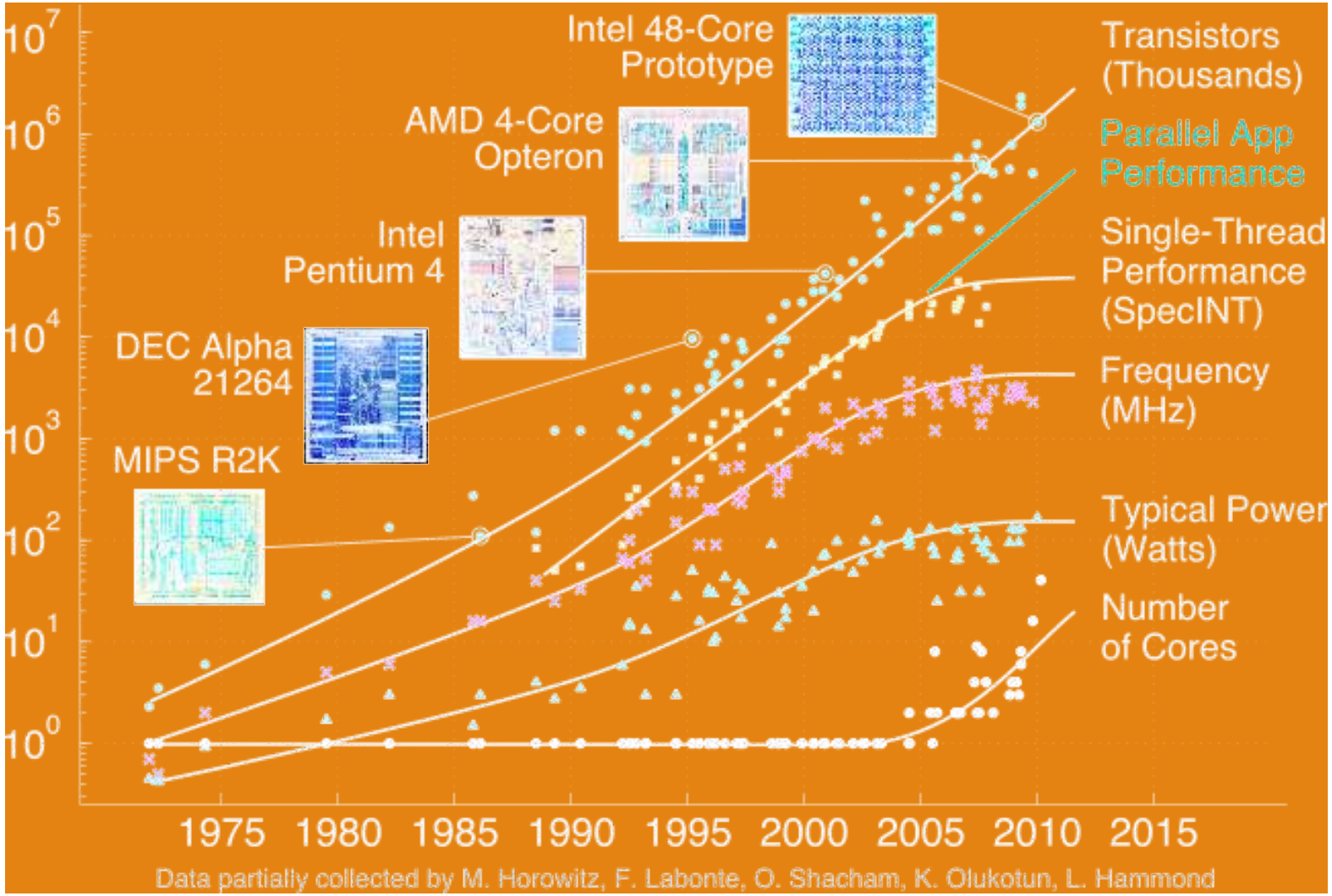


Transition to Multicore



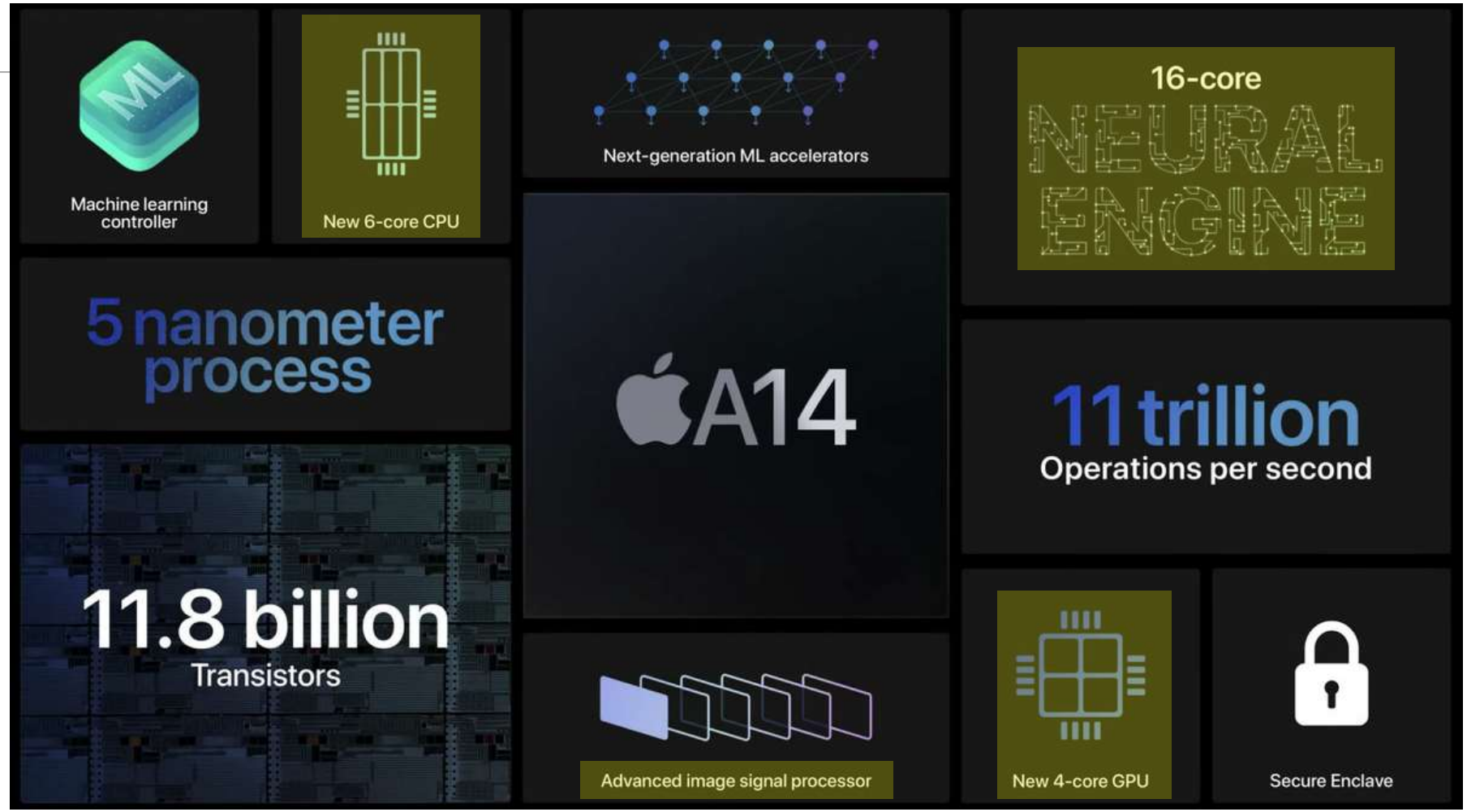
Sequential App Performance

Transition to Multicore



Sequential App Performance

Apple A14 Chip (a bit dated, but has core info)



Multiprocessor Execution Model

Each processor (core) executes its own instructions

Separate resources (not shared)
- Datapath (PC, registers, ALU)
- Highest level caches (e.g., 1st and 2nd)

Shared resources
- Memory (DRAM)
- Often 3rd level cache
  - Often on same silicon chip
  - But not a requirement

Nomenclature
- "Multiprocessor Microprocessor"
- Multicore processor
  - E.g., four core CPU (central processing unit)
  - Executes four different instruction streams simultaneously

Course Scope

Master the art of multitasking for your code! This course covers the fundamentals of parallel and distributed computing, from threading and shared memory to powerful message passing techniques. Learn to build efficient and scalable applications while gaining crucial skills for high-performance computing, big data, and beyond. Become a sought-after programmer with the ability to tackle complex challenges and take your career to the next level.

Week #	Course Plan
1	**Introduction to Parallel and Distributed Computing** History of computers, operating systems and sequential algorithms. Review: Types of processors (RISC & CISC) Flynn's taxonomy SISD, SIMD (vector and array processors), MIMD (shared and distributed memory), GPU
2	**Concurrent systems** Multitasking Systems, system API for multiprogramming Multithreading Systems, system APIs for thread management Multiprocessing Systems, shared memory architecture
3 & 4	**Concurrency Control** Mutual exclusion and synchronization System APIs for concurrency control
5 & 6	**Data Distribution Techniques** Inter process communications using PIPS/FIF/Shared Memory Network Sockets
7	**Parallel Random Access Machines** Overview of Parallel Random Access Machine, PRAM Models, EREW PRAM Algorithms Analysis of ERCW-PRAM, Algorithms
8	**Parallel Random Access Machines** Analysis of CRCW-PRAM Algorithms Analysis of CREW-PRAM Algorithms
9	**Distributed Computing** Cluster Computing, GRID Computing, Overview of Available tools, Overview of Message Passing Systems MPI/MPICH & Applications.
10	**Message Passing Interface (MPI)** Six Basic APIs to implement distributed memory applications APIs for group management and communications
11	**Message Passing Interface (MPI)** APIs for data distribution and collections Collective operations Converting PRAM models into MPI Programs
12	**General-Purpose Graphics Processor Architectures** GPU Architecture Algorithmic design for GPU Programming
13	**General-Purpose Graphics Processor Architectures** GPU Programming using CUDA-C/ CUDA-Python
14	**Project submission and Final Lab**

Criteria

- Final - 30 Marks
- Midterm - 20 Marks
- Projects - 15 Marks (will be discussed after third week)
- Quiz - 15 Marks (Min 10)
- Labs - 20 Marks

Administrative Rules/Policies

- **Well discipline** is expected from class. Don't bother class for attendance.
- To avoid disturbance to others, **Mobile phones** must be set to the silent mode. If you want to attend call or leave class for any reason just leave without asking same goes with entering in the class.
- We will have some **Pop-Quizzes** (means without prior announcement).
- **Late submissions** of assignments/Labs are subject to no submission at all.
- There will be no **deadline extension** in this course.
- **Copying** is strictly prohibited (Same submission will be marked zero).
- **Active involvement** in lectures will be highly appreciated.
- Communication medium is email only except for Class CR.

A Simple Tip For This Course!

- This course may not be simple at all so do attend lecture physically and mentally both.
- It will be beneficial for you; if you ensure, before you come to the new lecture, you understand the material presented during the earlier lectures.
- Your Aim should be :

  Focus on **LEARNING**, instead **GRADING**

  – If you learn, definitely, you will get **Good Grade**

  – If you target grades, it is more likely, you will not learn and even you will lose grades!

ANY QUESTION?

Why do we need parallelization?
Why don’t we make a better computer?

Units of Measure for HPC

High Performance Computing (HPC) units are:

- **Flop: floating point operation, usually double precision unless noted**
- **Flop/s: floating point operations per second**
- **Bytes: size of data (a double precision floating point number is 8 bytes)**

Typical sizes are millions, billions, trillions…

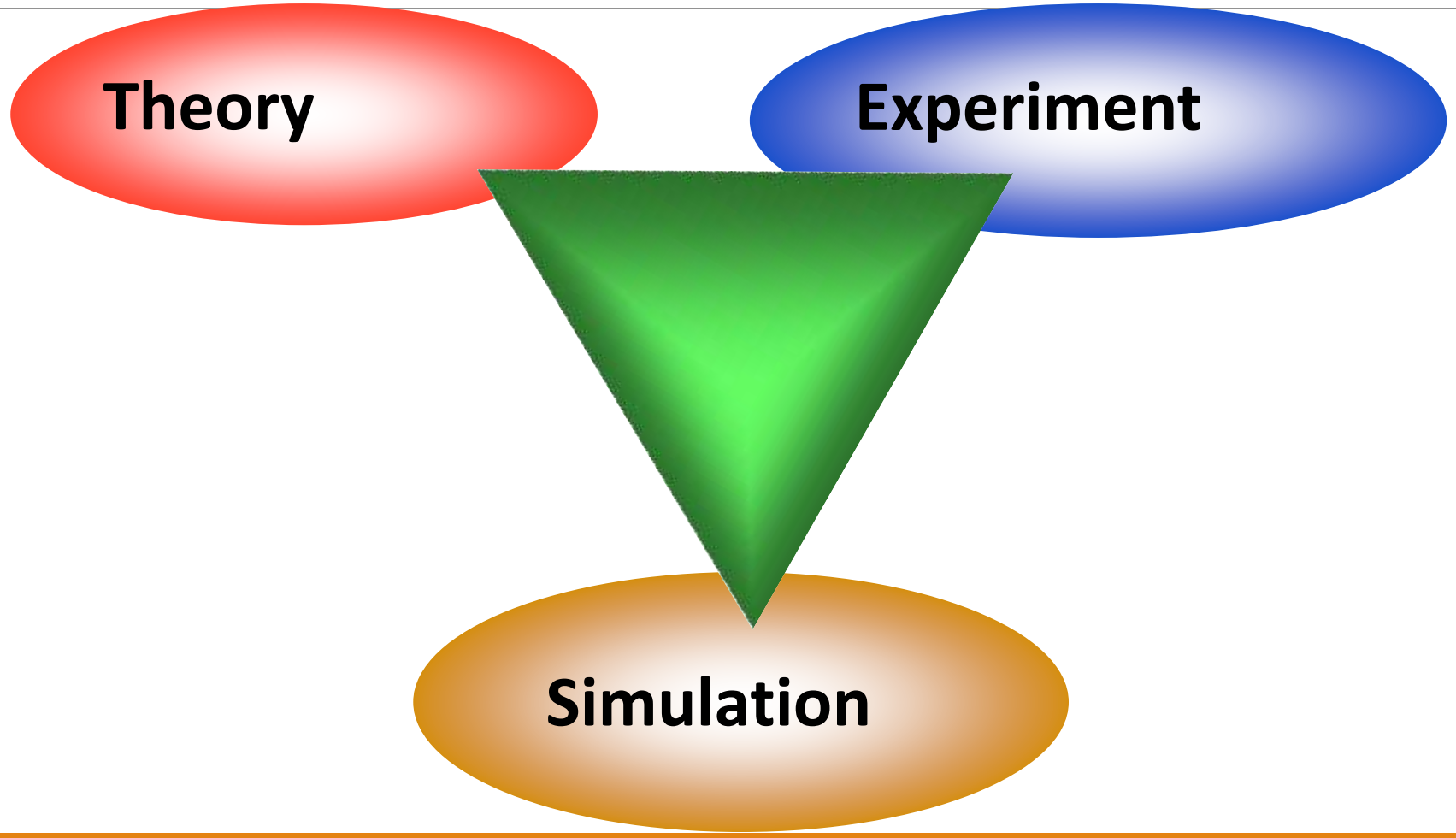
Kilo	Kflop/s = 10^3 flop/sec	Kbyte = $10^3 \sim 2^{10}$ = 1024 bytes (KiB)
Mega	Mflop/s = 10^6 flop/sec	Mbyte = $10^6 \sim 2^{20}$ bytes (MiB)
Giga	Gflop/s = 10^9 flop/sec	Gbyte = $10^9 \sim 2^{30}$ bytes (GiB)
Tera	Tflop/s = 10^{12} flop/sec	Tbyte = $10^{12} \sim 2^{40}$ bytes (TiB)
Peta	Pflop/s = 10^{15} flop/sec	Pbyte = $10^{15} \sim 2^{50}$ bytes (PiB)
Goal → Exa	Eflop/s = 10^{18} flop/sec	Ebyte = $10^{18} \sim 2^{60}$ bytes (EiB)
Zetta	Zflop/s = 10^{21} flop/sec	Zbyte = $10^{21} \sim 2^{70}$ bytes (ZiB)
Yotta	Yflop/s = 10^{24} flop/sec	Ybyte = $10^{24} \sim 2^{80}$ bytes (YiB)

Current fastest (public) machines are petaflop systems

- **Up-to-date list at www.top500.org**

Science using High Performance Computing

The “Third Pillar” of Science



Data analytics in science and engineering

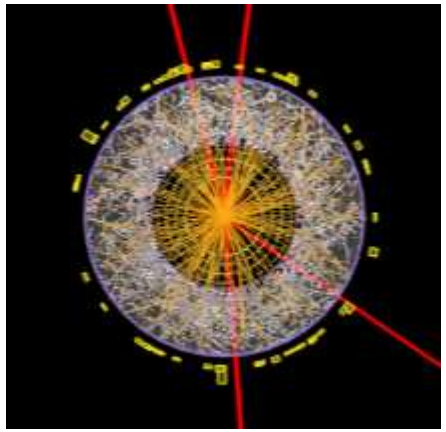
High Performance Data Analytics (HPDA) is used for data sets that are:

- too big
- too complex
- too fast (streaming)
- too noisy
- too heterogeneous

for measurement alone

Images from telescopes

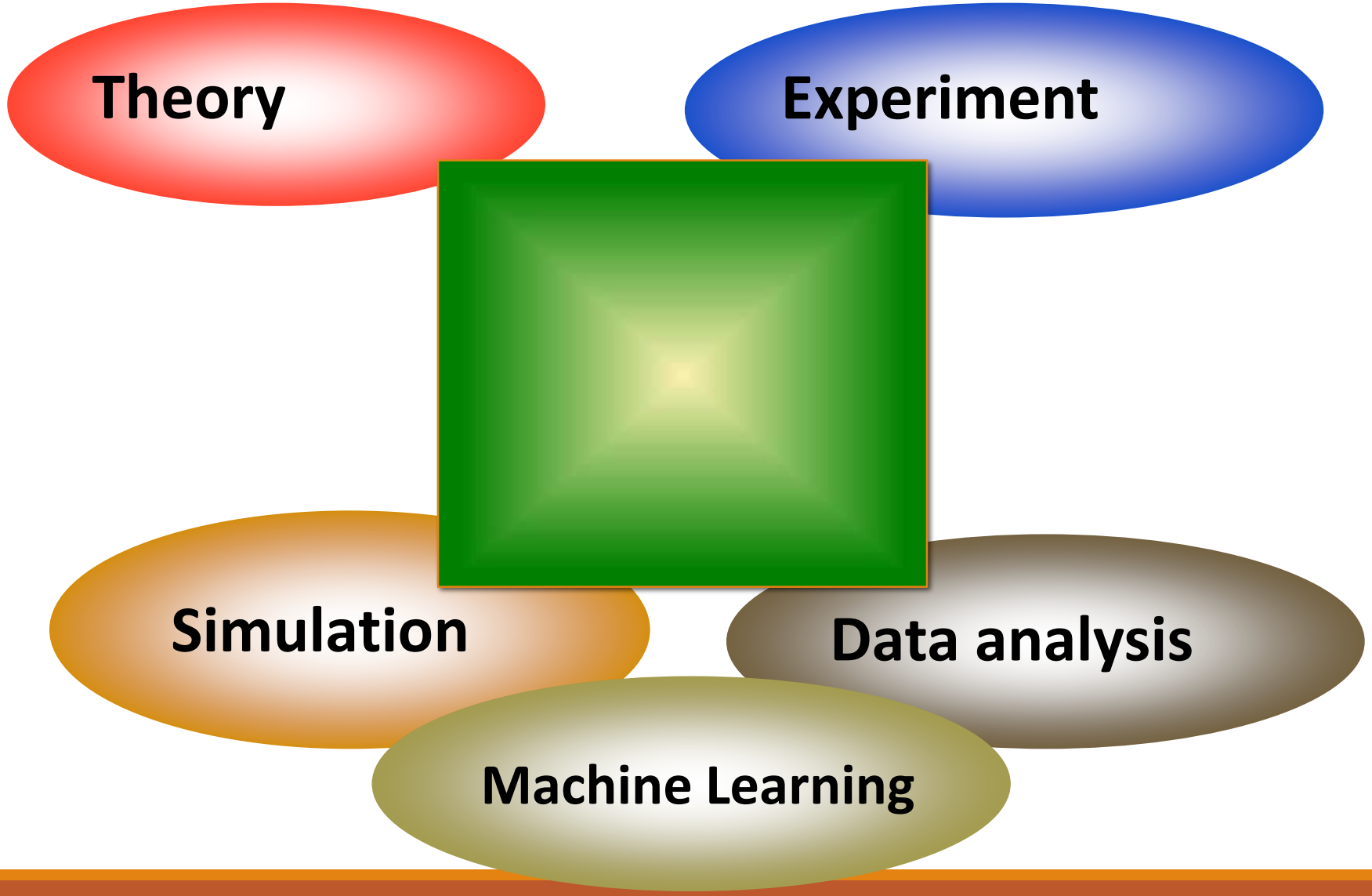
Genomes from sequencers



Particle from detectors

Sensor data

The Fourth Paradigm of Science



Simulation in Science and Engineering

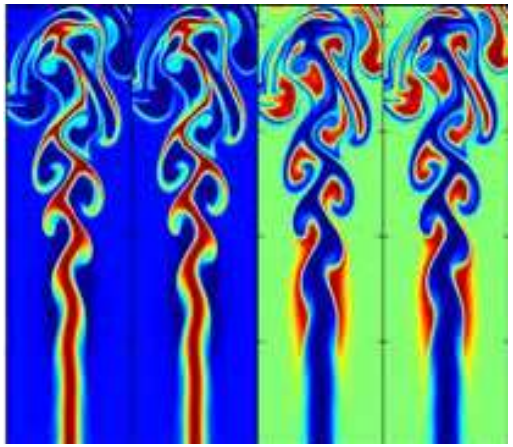
High performance computing (HPC) simulation to understand things that are:

- too big
- too small
- too fast
- too slow
- too expensive or
- too dangerous

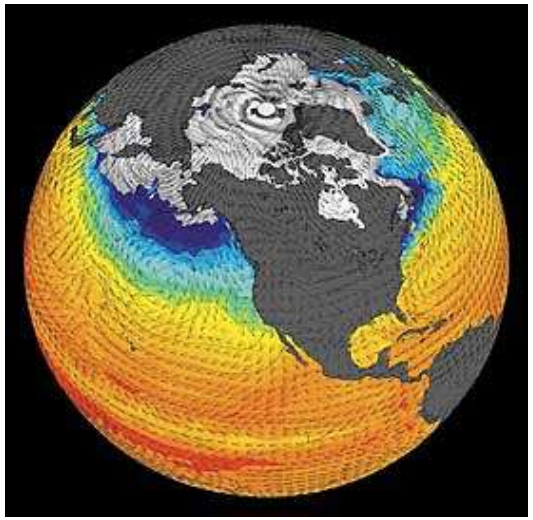
for experiments

Understanding the universe

Proteins and diseases

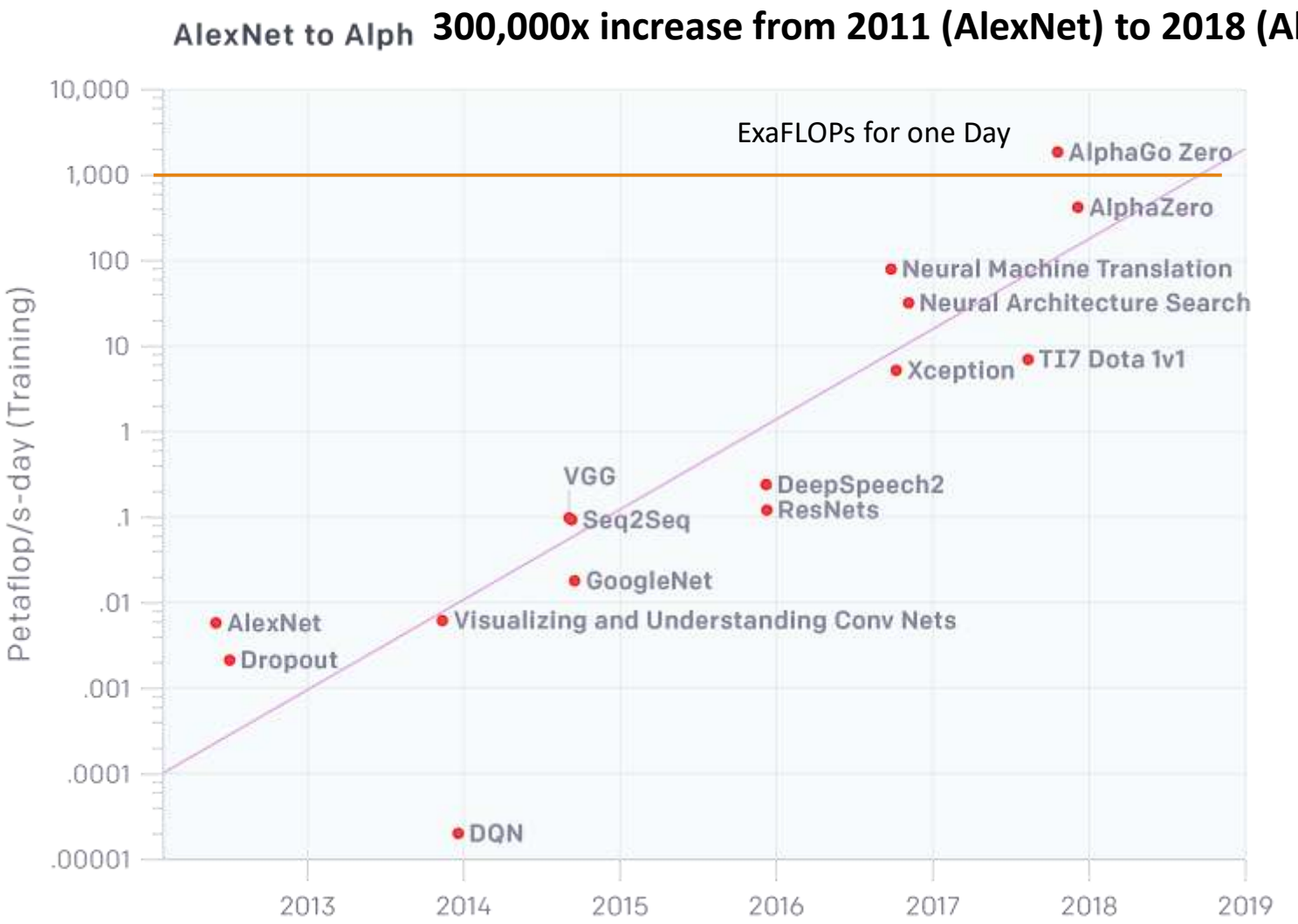


Energy-efficient jet engines



Climate change

Machine learning demands more computing



From 2011-2017 the fastest Top500 machine grew < 10x

https://blog.openai.com/ai-and-compute/

Why the ~~Fastest~~ all Computers are Parallel Computers

(since 2005)

INCLUDING LAPTOPS AND HANDHELDS

Some historical context: why avoid parallel processing?

- Single-threaded CPU performance doubling ~ every 18 months
- Implication: working to parallelize your code was often not worth the time
  - Software developer does nothing, code gets faster next year. Woot!

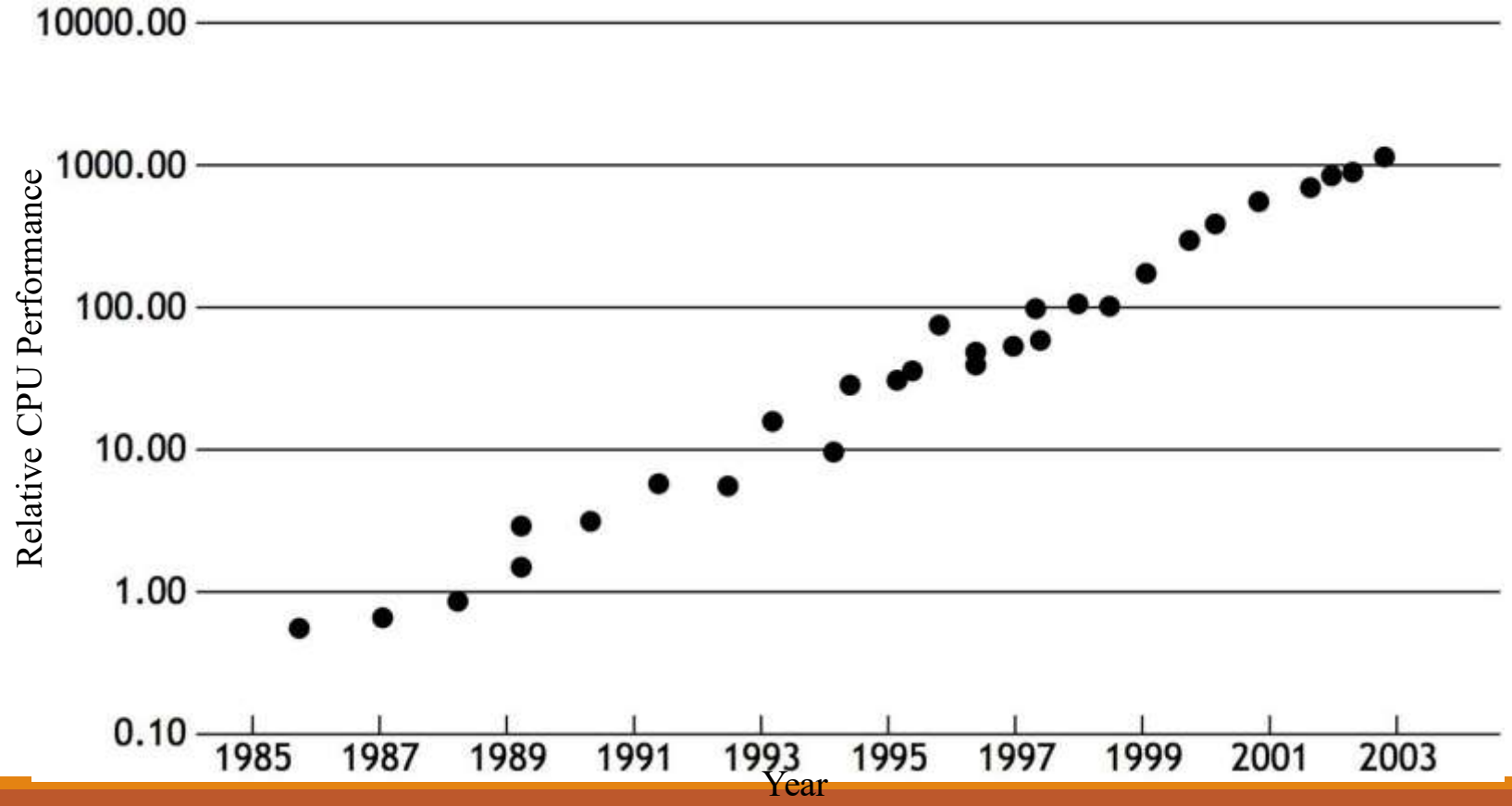


Image credit: Olukotun and Hammond, ACM Queue 2005

Until ~15 to 17 years ago: two significant reasons for processor performance improvement

1. **Exploiting instruction-level parallelism (superscalar execution)**

2. **Increasing CPU clock frequency**

What is a computer program?

Here is a program written in C

```
int main(int argc, char** argv) {

    int x = 1;

    for (int i=0; i<10; i++) {
       x = x + x;
    }

    printf(“%d\n”, x);

    return 0;
}
```

What is a program? (from a processor's perspective)

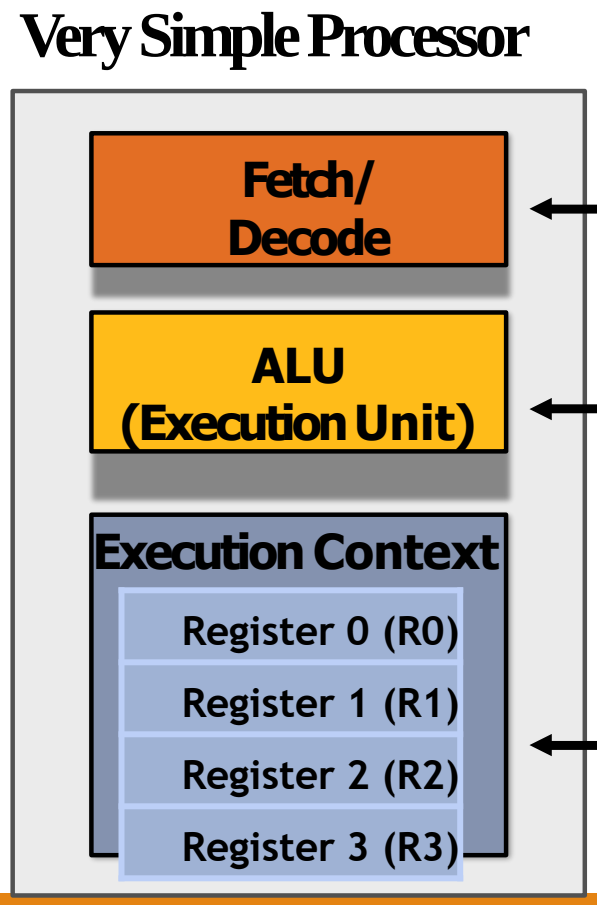
A program is just a list of processor instructions!

```c
int main(int argc, char** argv)
{
  int x = 1;

  for (int i=0; i<10; i++)
    { x = x + x;
  }

  printf("%d\n", x);

  return 0;
}
```

Compile code

```
_main:
100000f10:    pushq     %rbp
100000f11:    movq %rsp, %rbp
100000f14:    subq $32, %rsp
100000f18:    movl $0, -4(%rbp)
100000f1f:    movl %edi, -8(%rbp)
100000f22:    movq %rsi, -16(%rbp)
100000f26:    movl $1, -20(%rbp)
100000f2d:    movl $0, -24(%rbp)
100000f34:    cmpl $10, -24(%rbp)
100000f38:    jge  23 <_main+0x45>
100000f3e:    movl -20(%rbp), %eax
100000f41:    addl -20(%rbp), %eax
100000f44:    movl %eax, -20(%rbp)
100000f47:    movl -24(%rbp), %eax
100000f4a:    addl $1, %eax
100000f4d:    movl %eax, -24(%rbp)
100000f50:    jmp  -33 <_main+0x24>
100000f55:    leaq 58(%rip), %rdi
100000f5c:    movl -20(%rbp), %esi
100000f5f:    movb $0, %al
100000f61:    callq     14
100000f66:    xorl %esi, %esi
100000f68:    movl %eax, -28(%rbp)
100000f6b:    movl %esi, %eax
100000f6d:    addq $32, %rsp
100000f71:    popq %rbp
100000f72:    rets
```

A processor executes instructions



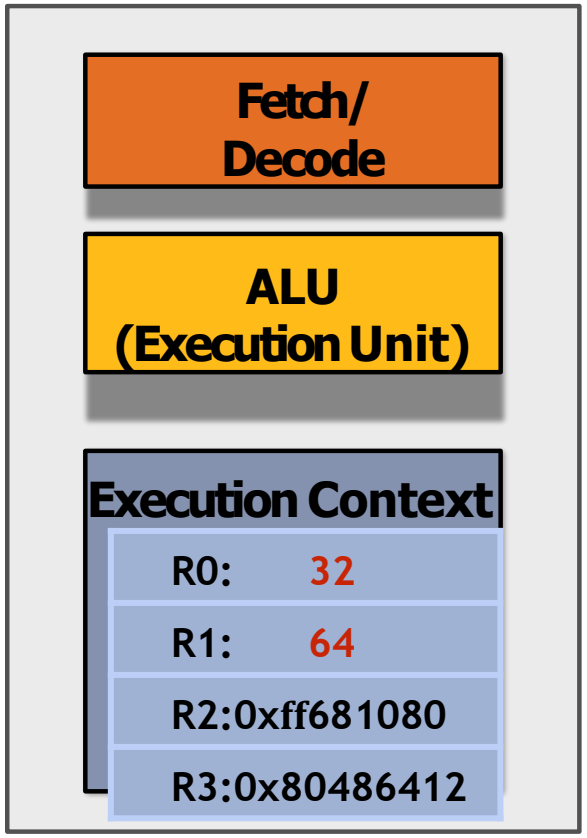
Determine what instruction to run next

Execution unit: performs the operation described by an instruction, which may modify values in the processor's registers or the computer's memory

Registers: maintain program state: store value of variables used as inputs and outputs to operations

One example instruction: add two numbers

Very Simple Processor



Step 1:

Processor gets next program instruction from memory (figure out what the processor should do next)

`add R0 ← R0, R1`

"Please add the contents of register R0 to the contents of register R1 and put the result of the addition into register R0"

Step 2:

Get operation inputs from registers

Contents of R0 input to execution unit: 32

Contents of R1 input to execution unit: 64

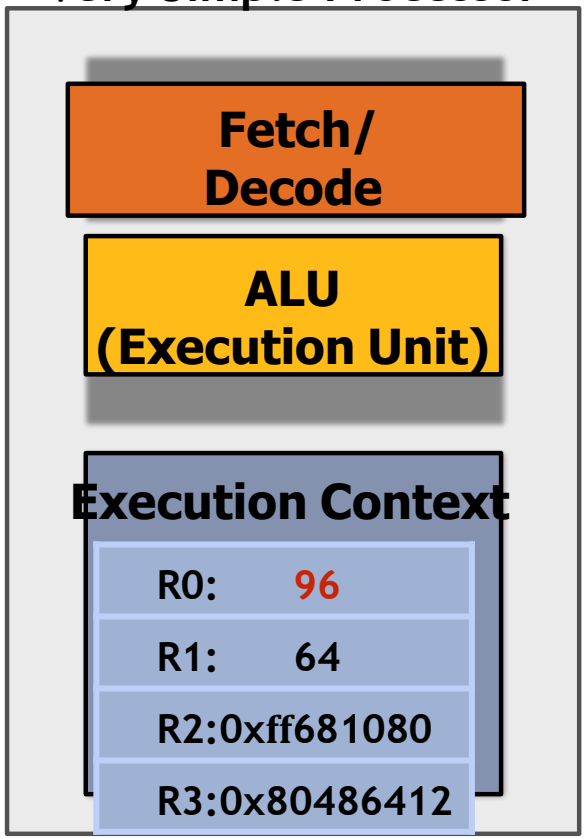
Step 3:

Perform addition operation:

Execution unit performs arithmetic, the result is: 96

One example instruction: add two numbers

Professor Kayvon's
Very Simple Processor



Step 1:
Processor gets next program instruction from memory (figure out what the processor should do next)

`add R0 ← R0, R1`

"Please add the contents of register R0 to the contents of register R1 and put the result of the addition into register R0"

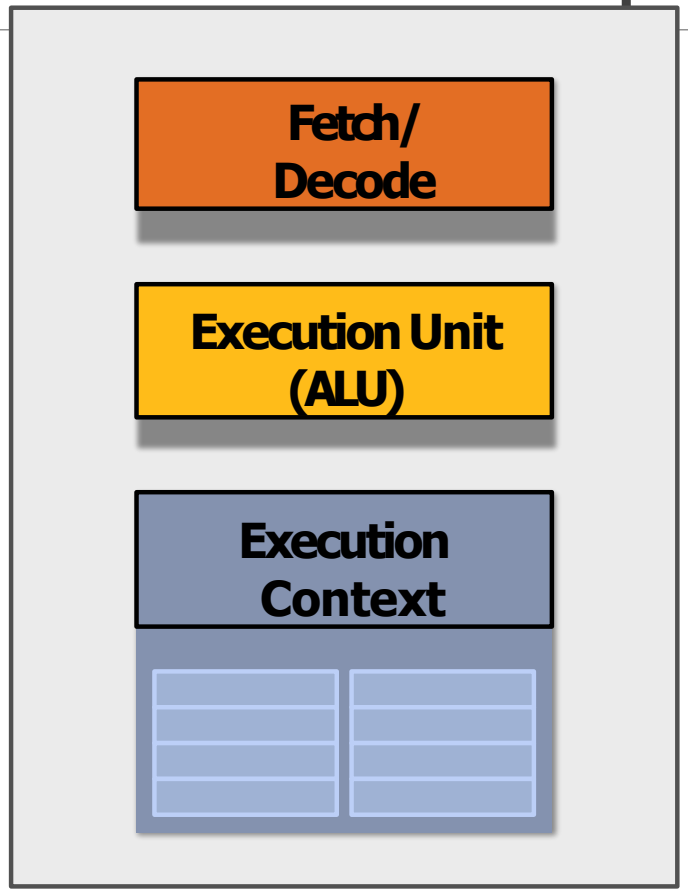
Step 2:
Get operation inputs from registers
Contents of R0 input to execution unit: 32
Contents of R1 input to execution unit: 64

Step 3:
Perform addition operation:
Execution unit performs arithmetic, the result is: 96

Step 4:
Store result 96 back to register R0

Execute program

My very simple processor executes one instruction per clock




```
ld    r0, addr[r1]
mul       r1, r0, r0
mul       r1, r1, r0
...
...
...
...
...
...
st    addr[r2], r0
```

Let's consider a very simple piece of code

```
a = x*x + y*y + z*z
```

Consider the following five instruction program:

Assume register R0 = x, R1 = y, R2 = z

```
mul R0, R0, R0
mul R1, R1, R1
mul R2, R2, R2
add R0, R0, R1
add R3, R0, R2
```

R3 now stores value of program variable 'a'

This program has five instructions, so it will take five clocks to execute, correct?

Can we do better?

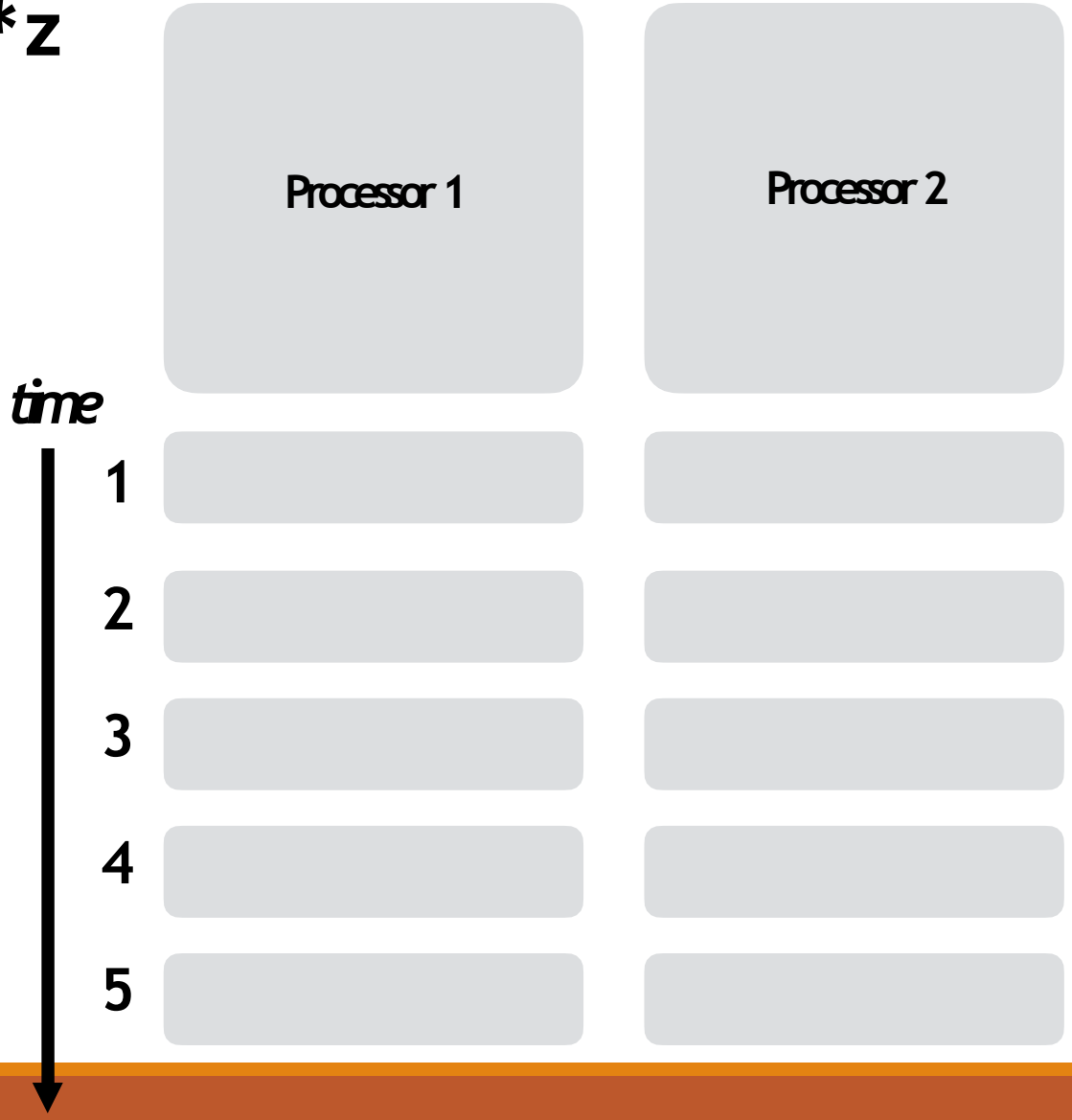
What if up to two instructions can be performed at once?

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'



What if up to two instructions can be performed at once?

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

time	Processor 1	Processor 2
1	1. mul R0, R0, R0	2. mul R1, R1, R1
2	3. mul R2, R2, R2	4. add R0, R0, R1
3	5. add R3, R0, R2	
4		
5		

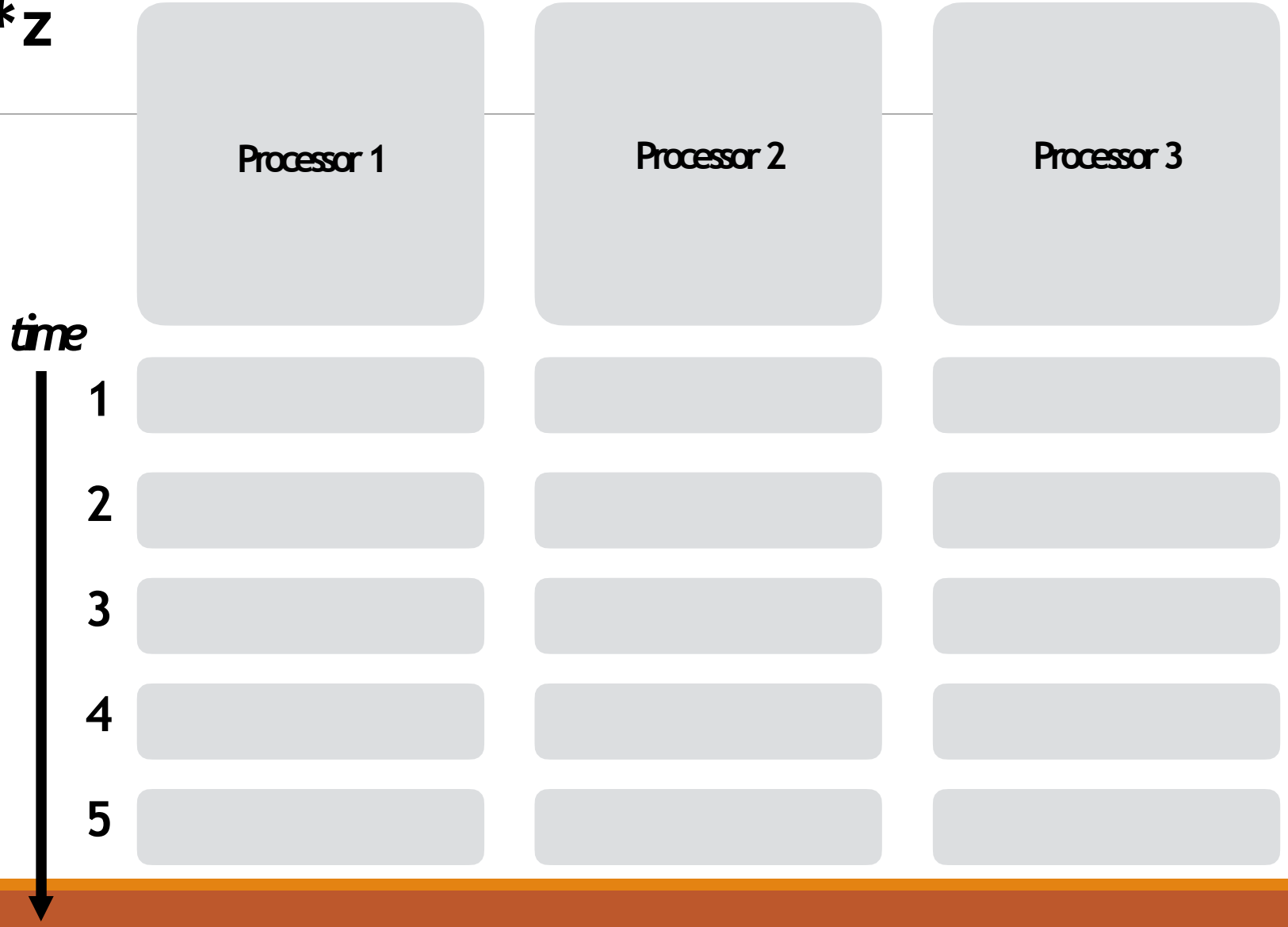
What about three instructions at once?

a = x*x + y*y + z*z

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'



What about three instructions at once?

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

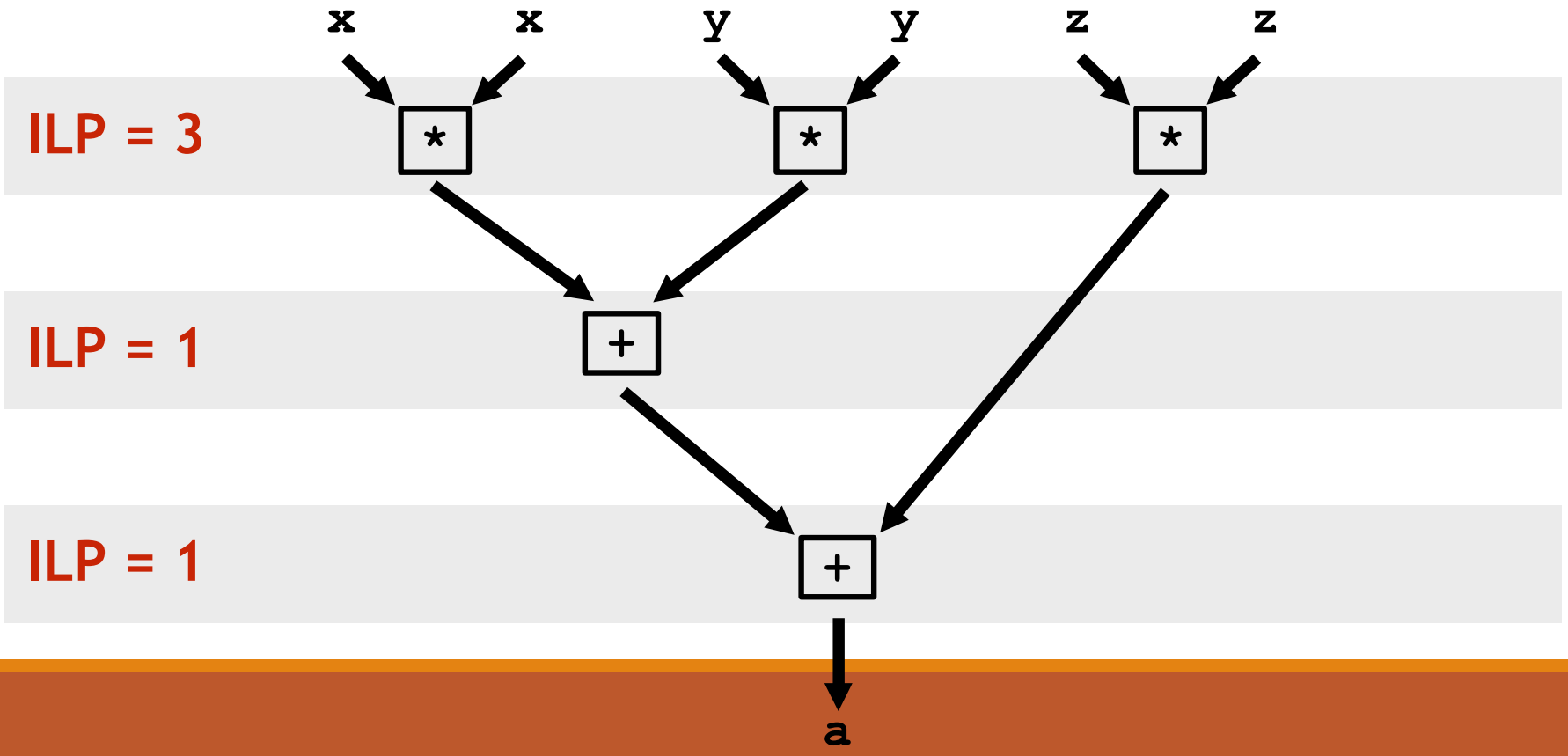
```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

time	Processor 1	Processor 2	Processor 3
1	1. mul R0, R0, R0	2. mul R1, R1, R1	3. mul R2, R2, R2
2	4. add R0, R0, R1		
3	5. add R3, R0, R2		
4			
5			

Instruction level parallelism (ILP) example

- ILP = 3 `a = x*x + y*y + z*z`



Superscalar processor execution

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

Idea #1:

Superscalar execution: processor automatically finds* independent instructions in an instruction sequence and executes them in parallel on multiple execution units!

In this example: instructions 1, 2, and 3 can be executed in parallel without impacting program correctness (on a superscalar processor that determines that the lack of dependencies exists)

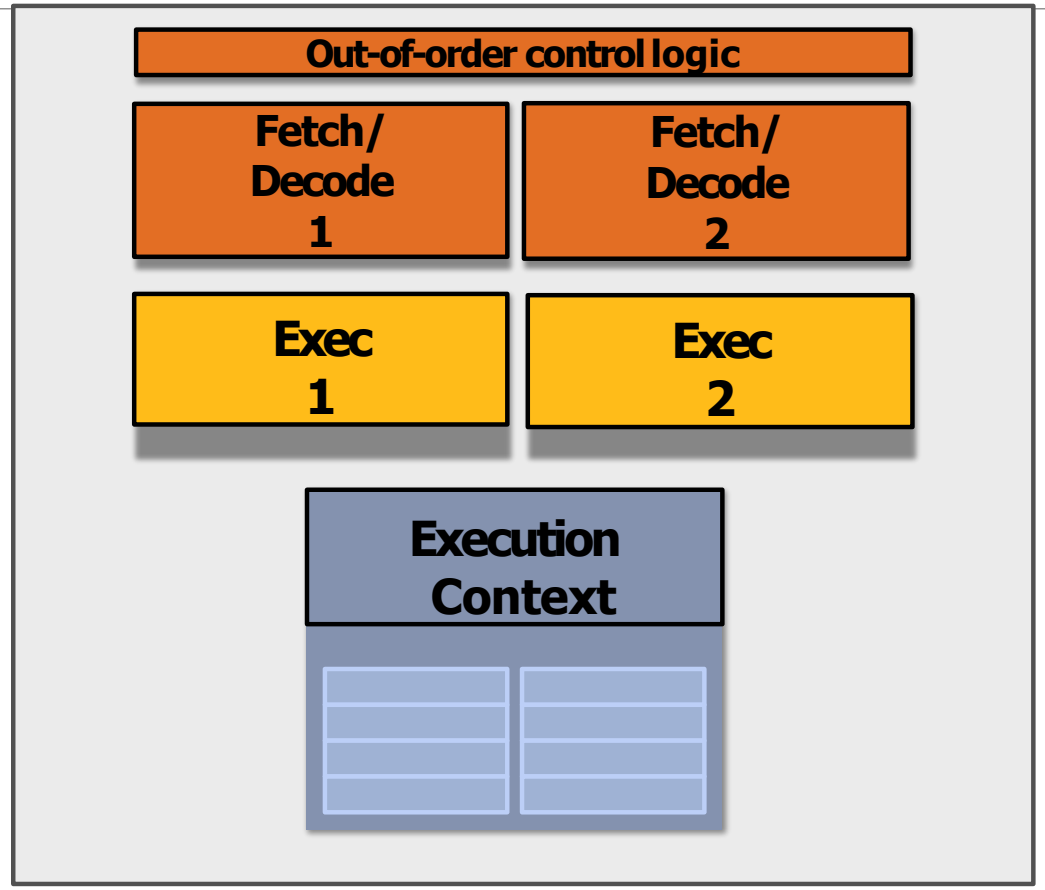
But instruction 4 must be executed after instructions 1 and 2

And instruction 5 must be executed after instruction 4

* Or the compiler finds independent instructions at compile time and explicitly encodes dependencies in the compiled binary.

Superscalar processor

This processor can decode and execute up to two instructions per clock



Aside:
Old Intel Pentium 4 CPU

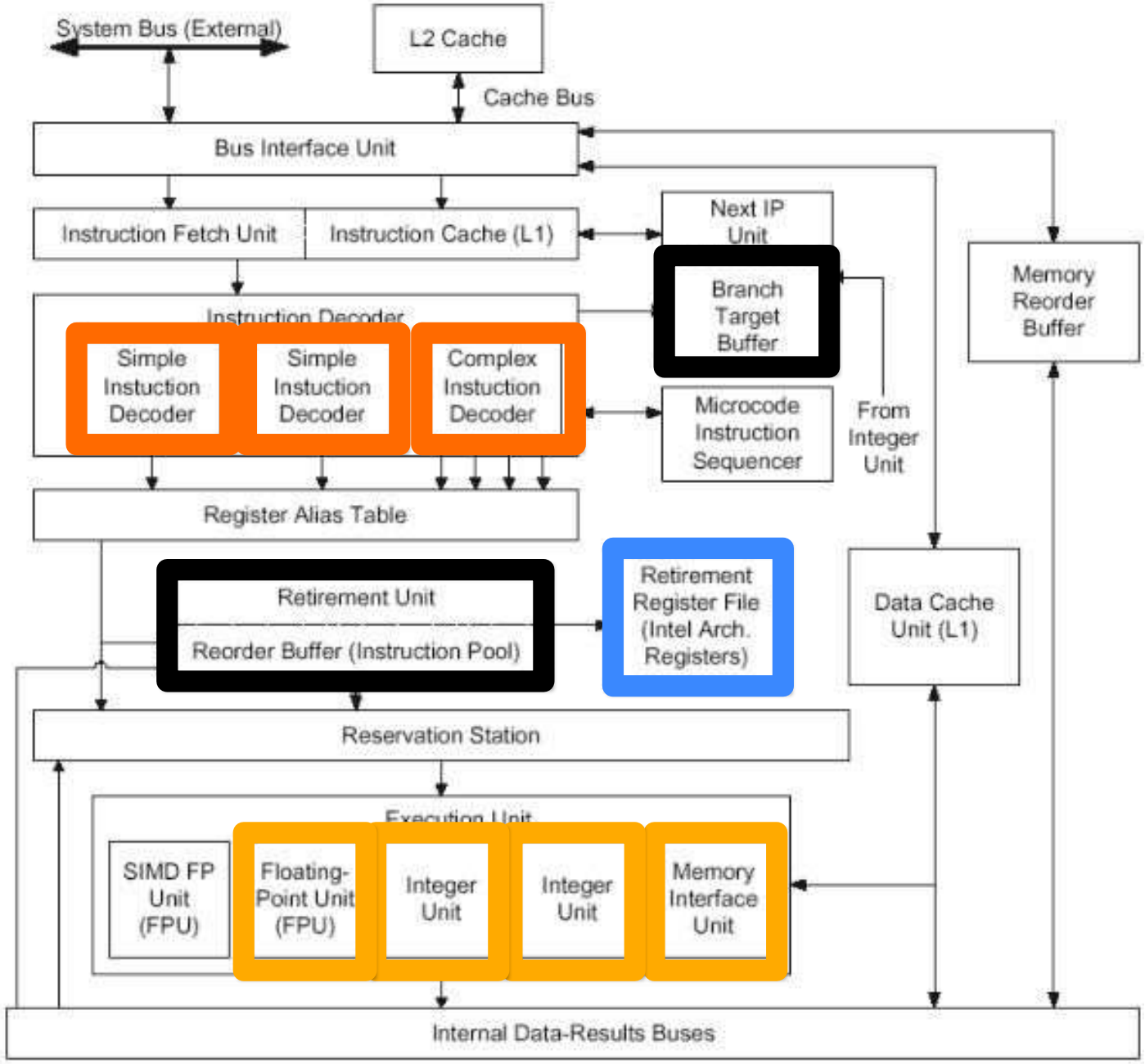


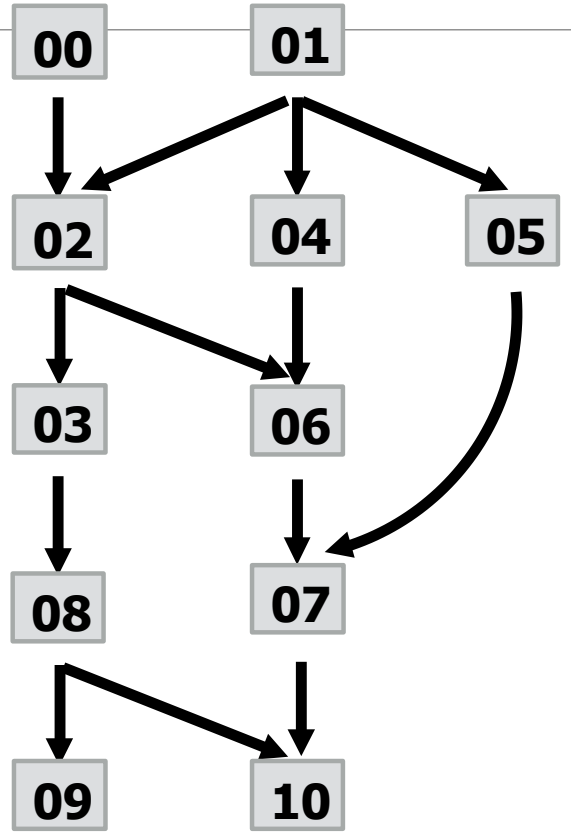
Image credit: http://xbitlabs.com/articles/pentium4/index.html

A more complex example

Program (sequence of instructions)

PC	Instruction	value during execution
00	a = 2	
01	b = 4	
2.	tmp2 = a + b	// 6
3.	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 1
		6
06	tmp6 = tmp + tmp4	// 1
	2	4
07	tmp7 = tmp + tmp6	// 3
	5	0

Instruction dependency graph

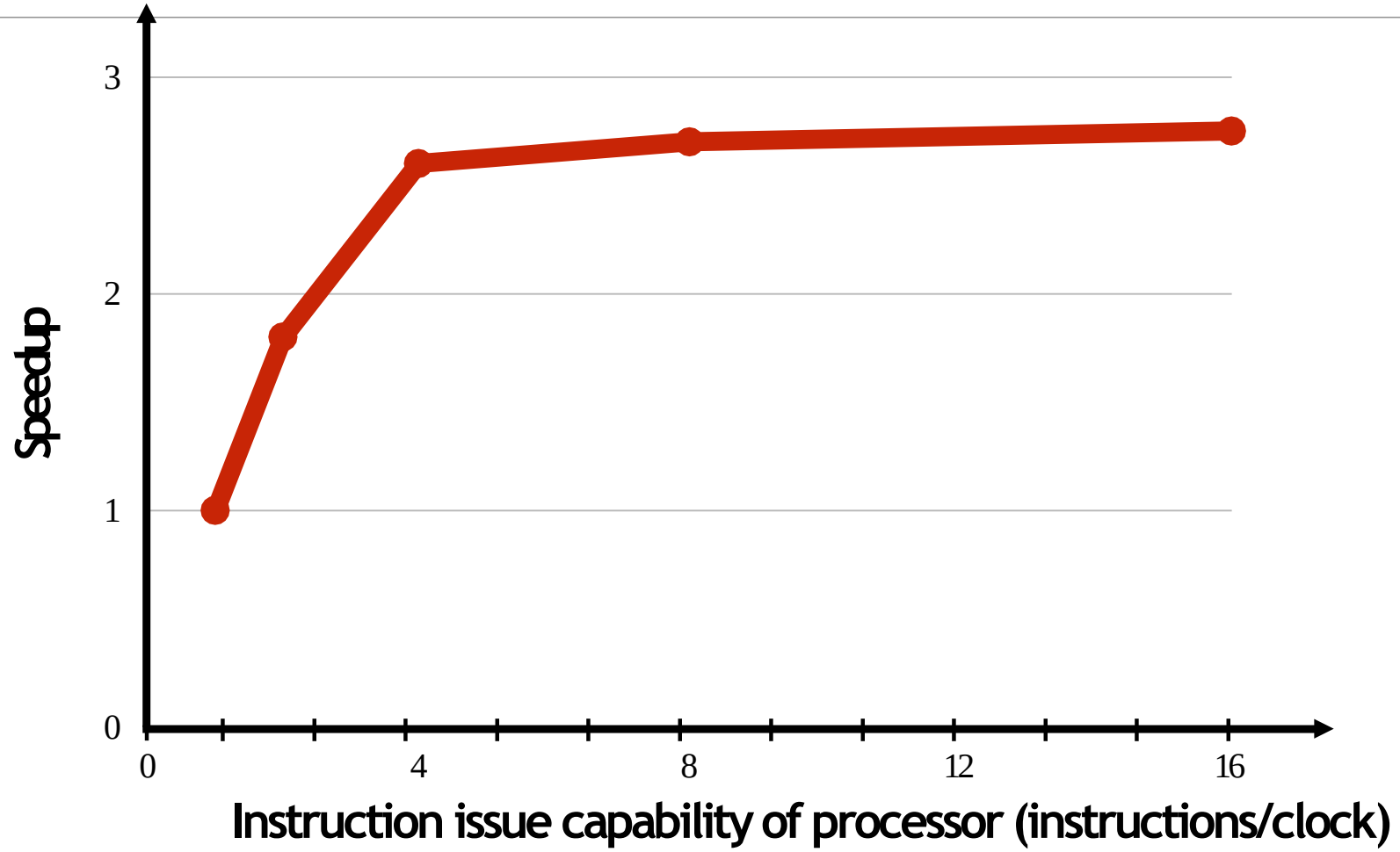


08 if

09 (

Diminishing returns of superscalar execution

Most available ILP is exploited by a processor capable of issuing four instructions per clock (Little performance benefit from building a processor that can issue more)

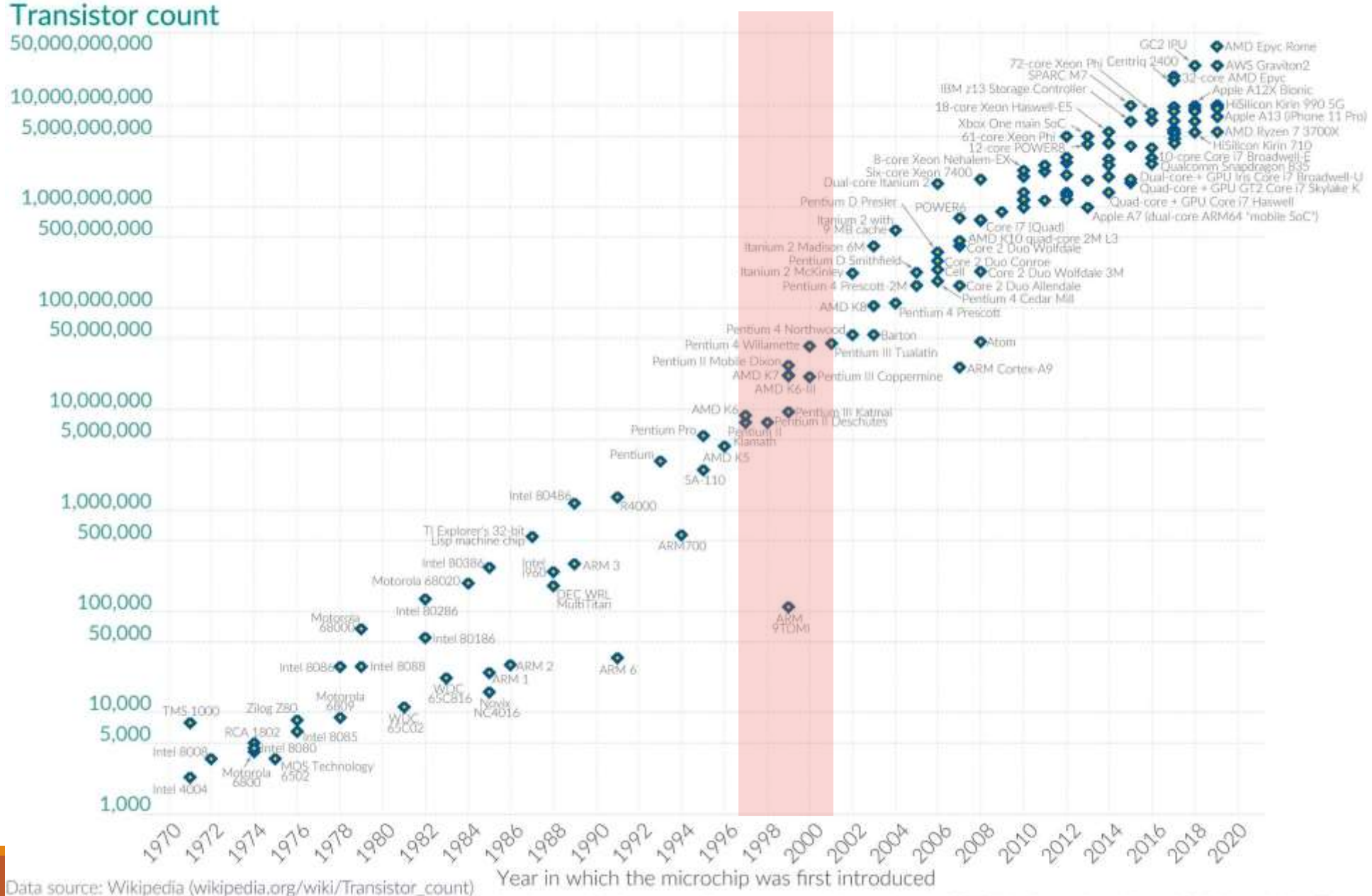


Source: Culler & Singh (data from Johnson 1991)

Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

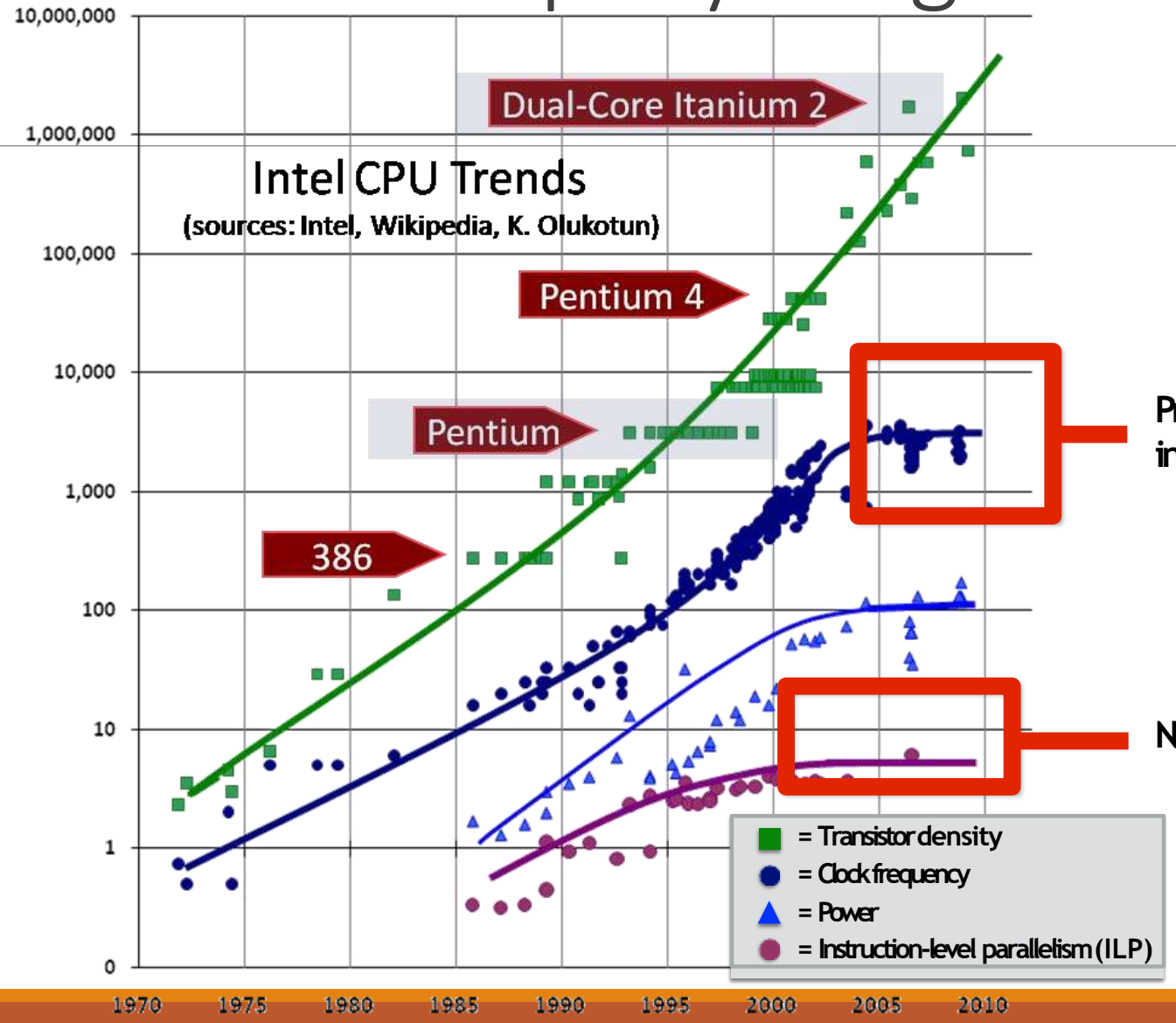


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

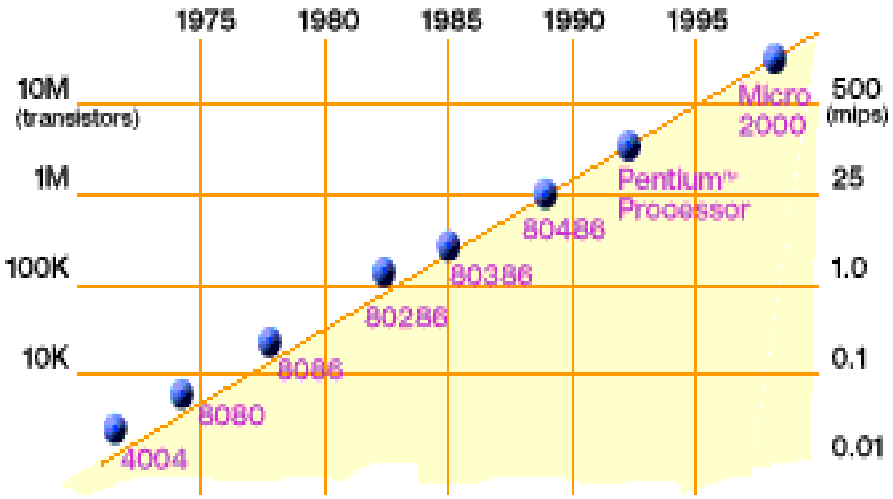
ILP tapped out + end of frequency scaling



Processor clock rate stops increasing

No further benefit from ILP

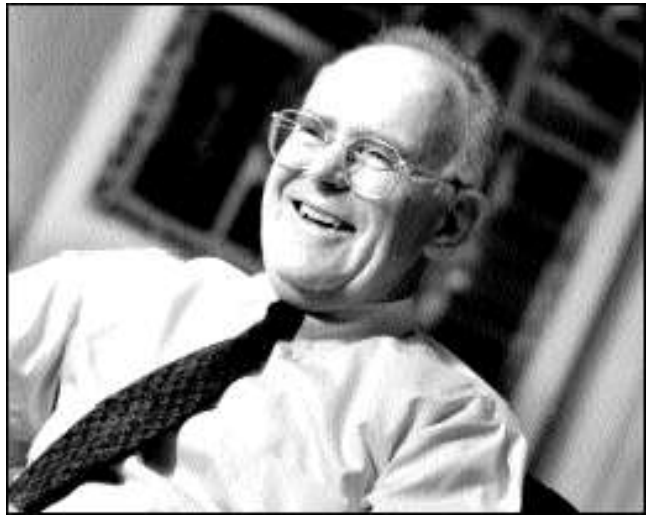
Technology Trends: Microprocessor Capacity



2X transistors/Chip Every 1.5 years

Called "Moore's Law"

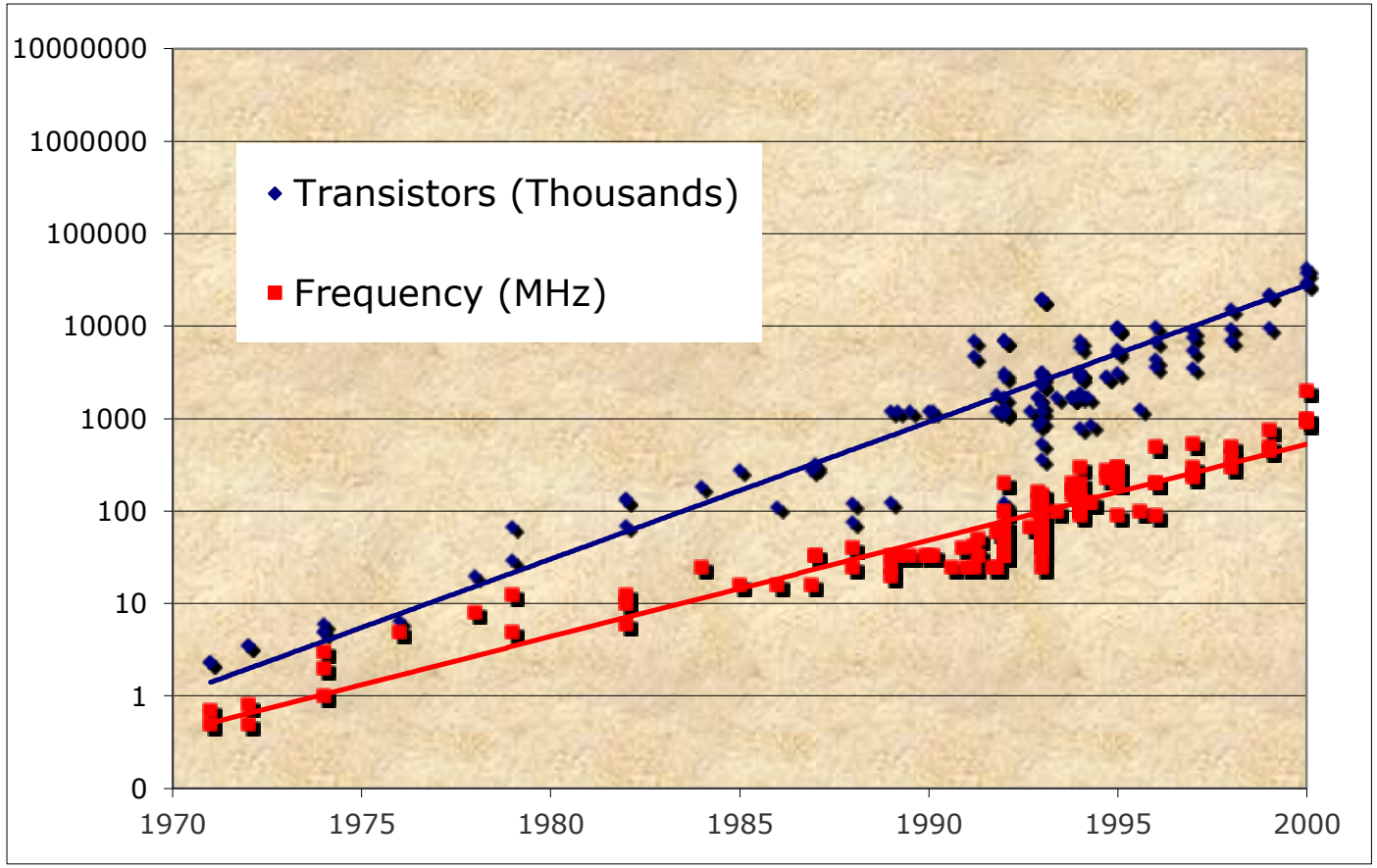
Microprocessors have become smaller, denser, and more powerful.



Gordon Moore (co-founder of Intel) predicted in 1965 that the transistor density of semiconductor chips would double roughly every 18 months.

Slide source: Jack Dongarra

Microprocessor Transistors / Clock (1970-2000)



Historical Impact of Device Shrinkage

What happens when the feature size (transistor size) shrinks by a factor of x ?

Clock rate goes up by x because wires are shorter

- actually, less than x, because of power consumption

Transistors per unit area goes up by x^2

Die size has also increased

- typically, another factor of ~x

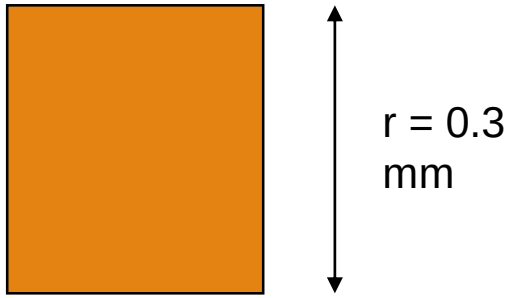
Raw computing power of the chip goes up by ~ x^4 !

- typically, x^3 is devoted to either on-chip
  - parallelism: hidden parallelism such as ILP
  - locality: caches

So, most programs x^3 times faster, without changing them

Limits: How fast can a serial computer be?

1 Tflop/s, 1 Tbyte sequential machine



Consider the 1 Tflop/s (10^{12}) sequential machine:

- Data must travel distance, r, from memory to processor.
- To get 1 data element per cycle, this means 10^{12} times per second at the speed of light, $c = 3x10^8$ m/s. Thus $r < c/10^{12} = 0.3$ mm.

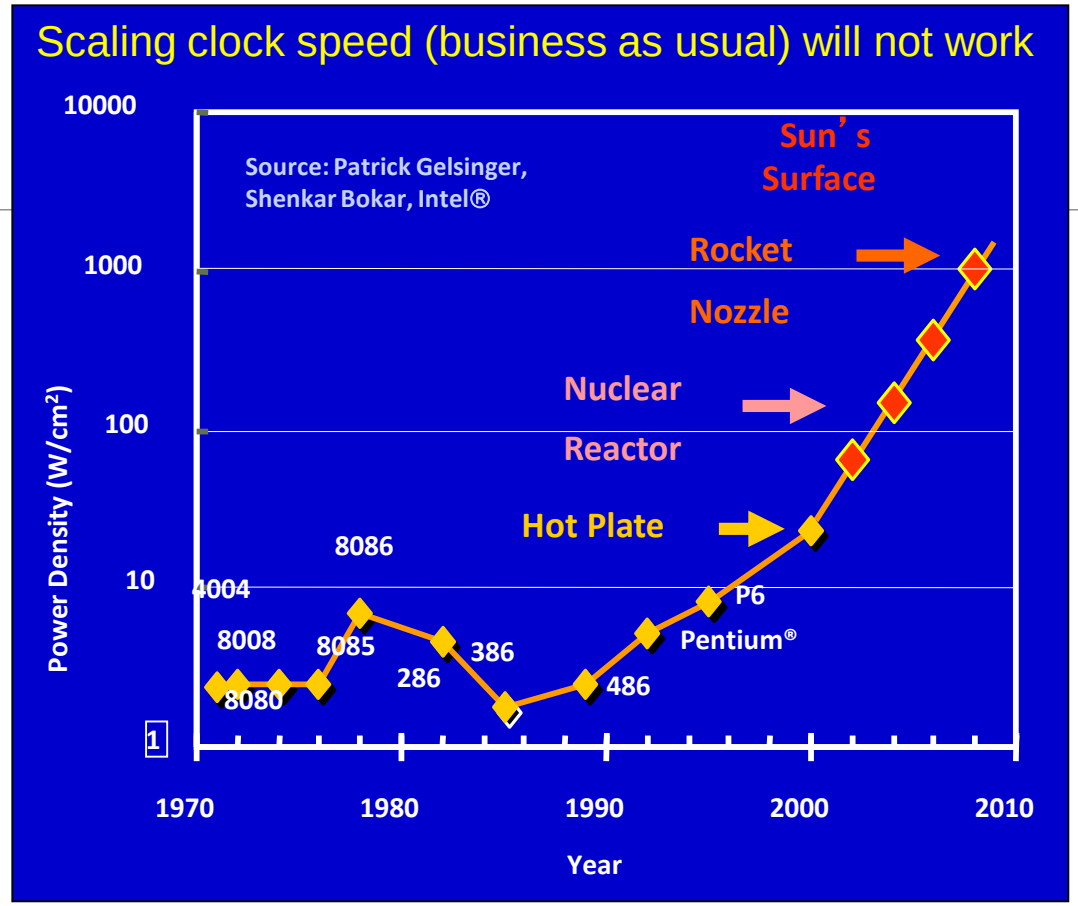
Now put 1 Tbyte of storage in a 0.3 mm x 0.3 mm area:

- Each bit occupies about 1 square Angstrom(10^{-10} meter), or the size of a small atom.

No choice but parallelism

Power Density Limits Serial Performance

- Concurrent systems are more power efficient
  - Dynamic power is proportional to V^2fC
  - Increasing frequency (f) also increases supply voltage (V) → cubic effect
  - Increasing cores increases capacitance (C) but only linearly
  - Save power by lowering clock speed



- High performance serial processors waste power
  - Speculation, dynamic dependence checking, etc. burn power
  - Implicit parallelism discovery
- More transistors, but not faster serial processors

A Brief History of Computing

- 1940s-1970s:
  - ENIAC, the first electronic general-purpose computer (1946).
  - Single-core CPUs and sequential programming models dominate.
  - Reference: "History of Computing Hardware" (Computer History Museum)
- 1970s-1990s:
  - Cray-1 supercomputer, a pioneer in vector processing (1976).
  - Rise of multicore processors and distributed systems.
  - Reference: "A History of Supercomputing" (IEEE Computer Society)
- 1990s-present:
  - Cluster computing and grid computing emerge.
  - Cloud computing democratizes access to distributed resources.
  - Reference: "The Evolution of Cloud Computing" (National Institute of Standards and Technology)

Instruction Set Architectures

Complex Instruction Set Computer (CISC)

- Major characteristics of a CISC architecture
  - A large number of instructions - typically from 100 to 250 instruction
  - Some instructions that perform specialized tasks are used infrequently
  - A large variety of addressing modes - typically from 5 to 20 different modes
  - Variable-length instruction formats
  - Instructions manipulate operands in memory

Reduced Instruction Set Computer (RISC)

- Major characteristics of a RISC architecture
  - Relatively few instructions and few addressing modes
  - Memory access limited to load and store instruction
  - All operations done within the registers of the CPU
  - Fixed-length, easily decoded instruction format
  - Single-cycle instruction execution
  - Hardwired rather than microprogrammed control

CISC

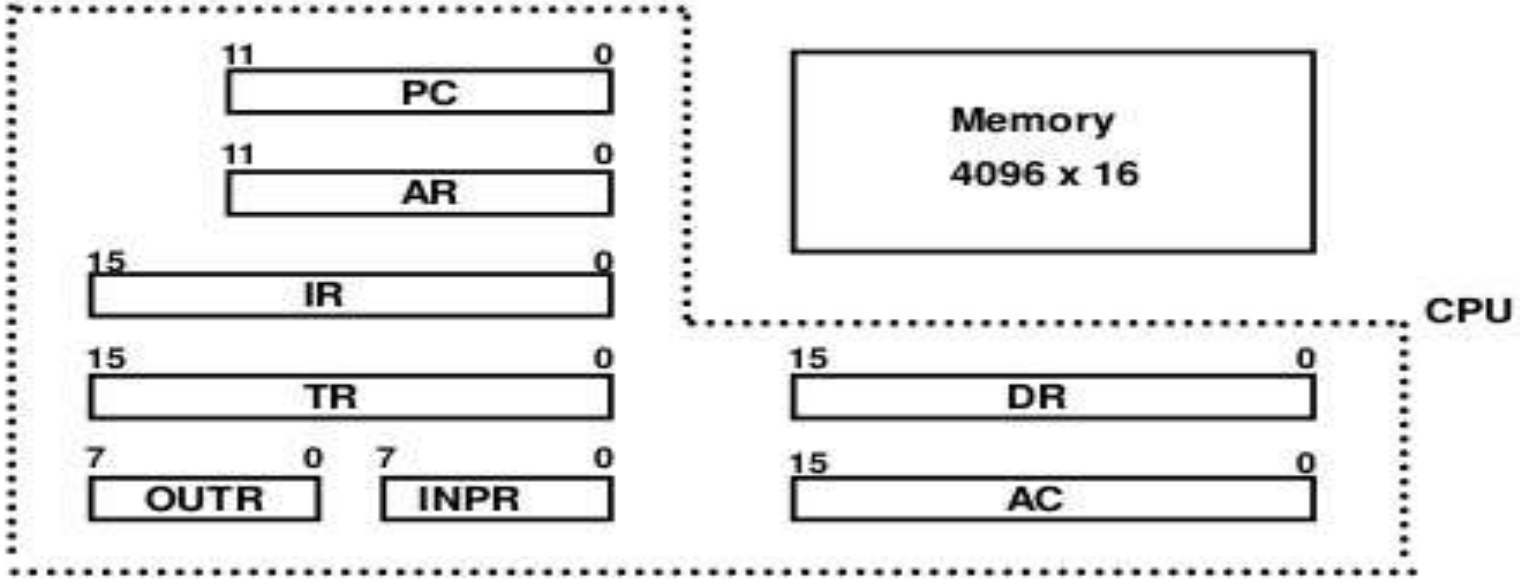
- x86 arch is a family of Complex instruction set computing (**CISC**) that is backward-compatible instruction set architectures (with variants of Intel 8086/8088)
  - majority of personal computers and laptops are based on the x86 architecture
- Register memory architecture allows operations to be performed on (or from) memory, as well as registers
  - In a register memory approach one of the operands for ADD operation may be in memory, while the other is in a register.
  - Example Model : Atmel microcontroller: AT89 series (Intel 8051 architecture)

RISC

- Advanced RISC Machine (ARM), is a family of reduced instruction set computing (**RISC**) architectures for computer processors, configured for various environments.
- Implement these architecture on: systems-on-chips (SoC) and systems-on-modules (SoM) that incorporate memory, interfaces, radios, etc.
- The only typical differentiating characteristic from CISC is that most RISC designs use uniform instruction length for almost all instructions, and employ strictly separate load/store-instructions.
- Comparing with X86, it improves cost, power consumption, and heat dissipation. As these characteristics are desirable for light, portable, battery-powered devices such as, smartphones, tablets, and other embedded systems.
  - With over 100 billion ARM processors produced as of 2017, ARM is the most widely used instruction set architecture in terms of quantity produced
  - Example Model : Atmel microcontroller: AT91SAM (ARM architecture)

BASIC COMPUTER REGISTERS

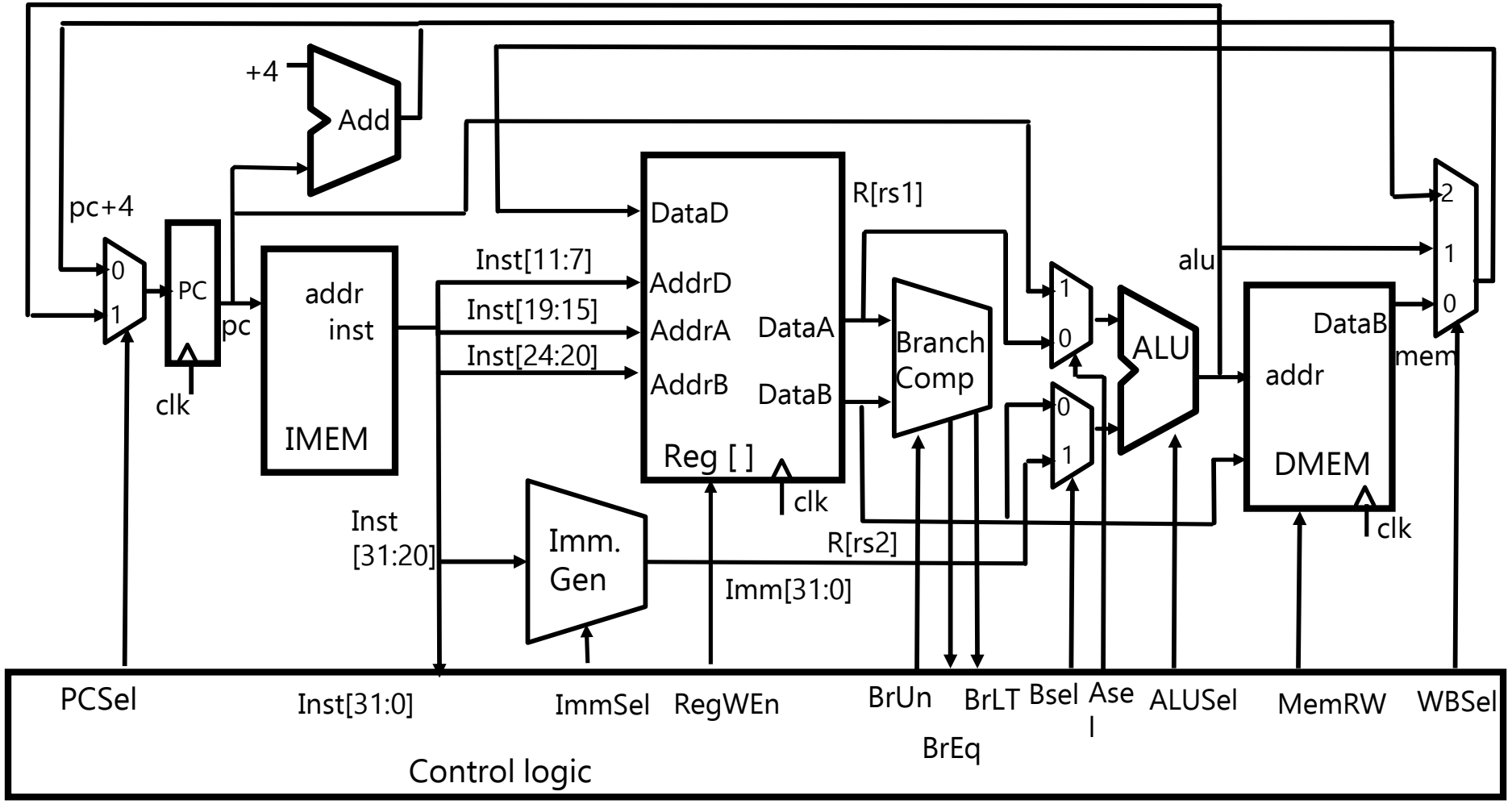
Registers in the Basic Computer



List of BC Registers

DR	16	Data Register	Holds memory operand
AR	12	Address Register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction Register	Holds instruction code
PC	12	Program Counter	Holds address of instruction
TR	16	Temporary Register	Holds temporary data
INPR	8	Input Register	Holds input character
OUTR	8	Output Register	Holds output character

Single-Cycle RV32I Datapath and Control



Resources used for this lecture

- https://gfxcourses.stanford.edu/cs149/fall22content/media/whyparallelism/01_whyparallelism.pdf